



UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE,
SCIENZE FISICHE E SCIENZE DELLA TERRA

Bachelor's Degree in Computer Science L31

Curriculum: Data Analysis

Comparison of Supervised vs. Unsupervised ResNet50 for Wildfires Detection

Supervisor:

Prof. Lorenzo Carnevale

Candidate:

Arya Khosravirad

Academic Year

2024/2025

Contents

1	Introduction	2
1.1	Context Definition	2
1.2	Motivation	2
1.3	Problem Identification	4
1.4	Proposed Solution	4
1.5	Objectives	5
1.6	Feasibility and Limitations	5
1.7	Structure of the Thesis	5
2	State of the Art	7
2.1	Overview of Fire Detection Systems	8
2.1.1	Human Based Observation	8
2.1.2	Satellite Systems	9
2.1.3	Optical Surveillance Systems	10
2.1.4	Wireless Sensor Networks (WSNs)	15
2.2	Why WSNs and Drones Go Well Together	17
2.2.1	Quick and Real-Time Detection:	17
2.2.2	Very Accurate for Early Detection:	17
2.2.3	Exact Location:	17
2.2.4	Works in All Weather:	17
2.2.5	Low Power and Self-Healing:	18

2.2.6	Comparison with Other Systems	18
2.3	Comparison of WSN-Based Fire Detection Systems:	19
2.3.1	Lloret et al. (Spain):	19
2.3.2	Son et al. (South Korea):	19
2.3.3	Hartung et al. (FireWxNet):	19
2.3.4	Alonso et al. (ZigBee Network for Fire Detection):	20
2.3.5	Which WSN to use?	21
2.3.6	ZigBee architecture	22
2.4	Principles and Methods of Fire Suppression	23
2.4.1	Understanding and Breaking the Fire Triangle	23
2.4.2	How Fire Suppression Works	24
2.5	Fire Suppression Techniques Overview	25
2.5.1	Direct Attack	25
2.5.2	Indirect Attack	26
2.5.3	Flank Attack	27
2.5.4	Parallel Attack	27
2.5.5	Conclusion	28
2.5.6	Why Night Containment with UAVs Is Useful	28
2.6	Comparing night containment drones with helicopters and airtankers	29
2.6.1	Cost context	29
2.6.2	Conclusion (cost and beyond)	34
2.7	Selecting the Optimal Foam for Drone Fire Suppression	36
2.7.1	Synthetic Foams (S1, S2, S3)	36
2.7.2	Aqueous Film-Forming Foams (AFFF)	36
2.7.3	Fluoroprotein Foams (FP1, FP2, FP3, FP4, FP5 AR)	37
2.7.4	Film-Forming Fluoroprotein Foams (FFFP)	37
2.7.5	Protein-Based Foams (P)	37
2.7.6	Conclusion	38

3	Material and Methods	39
3.1	Architecture	39
3.1.1	Assumptions	40
3.1.2	Proposed Components	42
3.2	Dataset	43
3.3	Pipelines	44
3.4	Preprocessing	46
3.4.1	Shared	47
3.4.2	Supervised only	49
3.4.3	Unsupervised only	52
3.5	Models	53
3.5.1	Shared:	53
3.5.2	Supervised only	54
3.5.3	Unsupervised only	56
3.6	Training	59
3.6.1	Supervised only	59
3.6.2	Unsupervised only	61
3.7	Evaluation:	67
3.7.1	Supervised only:	67
3.7.2	Unsupervised only:	70
4	Results and Discussion	79
4.1	Testbed	79
4.2	Experiments Plan	80
4.2.1	Threshold Tuning	80
4.2.2	Supervised	80
4.2.3	Unsupervised	83
4.3	Metrics	87
4.3.1	Before Threshold Tuning	87

4.3.2	After Threshold Tuning	88
4.4	My Consideration	89
5	Conclusion and Future Works	91

List of Figures

1.1	Burned areas in Sicily during 2024, Source: ISPRA [1].	3
2.1	Example of a human based forest fire observation tower	8
2.2	Example of a Satellite System capturing flames	9
2.3	Example of a Video Camera for forest fire observation	10
2.4	Example of an IR Camera for capturing flames	11
2.5	Example of a IR spectrometer for fire detection	12
2.6	Example of a LIDAR for forest fire detection	13
2.7	Example of ZigBee based wireless sensor network mesh for forest fire de- tection	22
2.8	Three fire triangle elements	24
2.9	Example of Direct attack	25
2.10	Example of Indirect Attack	26
2.11	Example of Flank Attack	27
2.12	Contract aircraft expenditures according to the USDA Forest Service 2022 Aviation Annual Report.	30
3.1	Proposed architecture.	40
3.2	Simplified activity diagram of how drones work using WSNs	41
3.3	Component diagram of how drones work using WSNs	42
3.4	Dataset layout. <code>train/</code> is labeled via subfolders and filenames. <code>test/</code> is flat with labels in filenames.	43
3.5	Dataset sample images	44

3.6	Side-by-side comparison of supervised and unsupervised wildfire pipelines.	46
3.7	File-type check confirming that Train Fire, Train No-Fire, and Test each contain only JPG images.	48
3.8	Training augmentation visualization: top row shows originals and bottom row shows randomly augmented versions.	55
3.9	Embedding extraction log	59
3.10	Training progress across 12 epochs	60
3.11	Training/validation curves: accuracy rises and loss falls, with close train-validation traces indicating good generalization and minimal overfitting.	61
3.12	Scree plot of PCA explained variance showing an elbow around 9 components, which are selected for projection.	62
3.13	Console output confirming PCA fitted on 1,515 training samples and projection to 9 dimensions for both Train and Test.	63
3.14	Elbow curve on training embeddings: the largest SSE drop is from k=1 to k=2 and the curve flattens afterward, indicating k=2.	63
3.15	Silhouette scores on training embeddings peak at k=2.	64
3.16	t-SNE (2D) of training embeddings.	65
3.17	t-SNE (3D) of training embeddings.	65
3.18	Random training examples per K-Means cluster (k=2), showing clear visual separation between non fire and fire scene.	66
3.19	Supervised classification report at threshold 0.5	68
3.20	Confusion matrix on supervised model with threshold 0.5.	69
3.21	Internal clustering metrics.	71
3.22	Cluster→class mapping learned on training data by majority vote.	72
3.23	Classification report for the unsupervised model at t=0:	74
3.24	Confusion matrix for the unsupervised model at t=0:	74
3.25	External clustering metrics	75
3.26	Images captured by Supervised demo	77
3.27	Images captured by Unsupervised demo	78

4.1	Chosen validation threshold for the supervised model.	81
4.2	Validation precision–recall (left, $\beta = 2.5$) and score histograms (right); the dashed line marks the tuned threshold ≈ 0.418 that favors recall. . .	82
4.3	Test classification report at tuned threshold	83
4.4	Confusion matrix (supervised, tuned)	83
4.5	Suggested threshold from training for the unsupervised	85
4.6	precision recall curve and score histogram for the unsupervised score; dashed line at $t \approx -0.081$ lies in the low-density gap and maximizes F_2	85
4.7	Test classification report for the tuned unsupervised model	86
4.8	Confusion matrix (unsupervised, tuned)	86
4.9	Supervised Confusion Matrix	88
4.10	Unsupervised Confusion Matrix	88
4.11	Supervised Confusion Matrix(tuned)	89
4.12	Unsupervised Confusion Matrix(tuned)	89

List of Tables

1.1	Burned forest area by province in Sicily in 2024(hectares). Source: ISPRA[1].	3
2.1	Comparison of Fire Detection Systems	16
2.2	Representative helicopter inputs for cost modeling.	30
2.3	Contract Helicopter Use and Costs Summary.	31
2.4	2022 Contract LAT/VLAT Use Summary by Aircraft Model	31
2.5	COTS UAV acquisition anchors (< USD 5,000) and flight time.	32
2.6	Firefighting Foam Parameters	38
4.1	Per-class metrics from classification reports — same test set (n=379) . .	88
4.2	Per-class metrics (tuned thresholds) on the same test set (n=379)	89

Listings

3.1	Dataset paths for training and testing	47
3.2	Sanity check: list unique file types per split	48
3.3	Filtering corrupted images	49
3.4	Supervised preprocessing — load images from a directory	50
3.5	Load Train/Fire (1) and Train/NoFire (0) with blur filtering	50
3.6	Concatenate class tensors into a unified training set	51
3.7	Load TEST images (keep filenames for reporting)	51
3.8	Train/Validation split with stratification (no leakage)	51
3.9	Listing and validating image file paths for training and testing datasets .	52
3.10	Image Loader with Resizing, BGR to RGB Conversion, and Quality Checks	53
3.11	Backbone definition	54
3.12	Supervised preprocessing: validation datagen and data flows	54
3.13	Custom classification head on top of the ResNet50 backbone	55
3.14	Model compilation with Adam optimizer and binary crossentropy loss . .	56
3.15	Extracting ResNet embeddings with tta mean using opencv loader	57
3.16	Training and evaluation data generators	58
3.17	Embedding Extraction (TTA for train, deterministic for test)	58
3.18	Model training with early stopping callback	60
3.19	Standardization of Training and Test Embeddings	61
3.20	Fitting PCA on Standardized Training Embeddings	62
3.21	Applying PCA Transformation and Normalization on Training and Test Sets	62

3.22	K-Means clustering on PCA-transformed Embeddings	64
3.23	Filtering and labeling test dataset images	67
3.24	Classification report and evaluation metrics	68
3.25	Internal Clustering Evaluation Metrics on Training and Test Sets	70
3.26	Inferring ground-truth labels from file paths and mapping clusters to classes	72
3.27	Compute signed class scores from K-Means centroid distances	73
3.28	Pre-tuning evaluation with closest-centroid threshold $t=0$ on TEST . . .	73
3.29	Demo function for simulating supervised version	76
3.30	Demo function for simulating Unsupervised version	77
4.1	Validation threshold via F_β maximization ($\beta = 2.5$)	81
4.2	Test evaluation with tuned threshold suggested	82
4.3	Cluster distance scoring and threshold selection using F_β measure	85
4.4	Test evaluation with tuned threshold	86

Abstract

Wildfires are becoming a serious issue, especially in the Mediterranean area. They destroy the environment, damage property, and most dangerously put human lives at risk. Traditional detection and firefighting methods, such as lookout towers, satellites, and planes, are often too slow, very costly, and sometimes unreliable [2].

To solve these problems, a system that uses drones to detect and support first responders to quickly suppress wildfires is proposed. The drones could be equipped with machine learning tools that can identify the first signs of a fire using real-time data collected from wireless sensors placed throughout the forest. As soon as a fire is detected, the drones, e.g., could go straight to the source of fire and release fire-suppressing foam, from a safe distance, particularly during night operations when crewed aircraft cannot fly. This strategy may be faster and more effective than most current techniques.

In this research, different fire-detection technologies are reviewed, followed by a focus on the training and evaluation of two image-based AI models. After the training phase, the fire detection models perform well on the selected fire images. In addition, a conceptual architecture that involves a wireless sensor network and drones for fire suppression are proposed. Finally, simple demonstrations of decision logic are included, without real deployment or simulations.

Acknowledgement

I want to start by thanking my supervisor, Prof. Lorenzo Carnevale from the bottom of my heart for all the help, patience and support he gave me while I was writing this thesis. He has been helpful and generous from the first conversations about my ideas to the last edits. His advice and helpful criticism made my research much better and also taught me how to think clearer and more confident when I face problems.

I also want to thank the University of Messina's Department of Mathematics and Computer Sciences, Physical Sciences, and Earth Sciences (MIFT) for giving me the chance to study in a good environment. The resources, classes and academic help I've gotten over the years have been very important in helping me to learn and grow.

I thank the Protezione Civile della Regione Sardegna for guidance that informed the night-containment operational concept.

Chapter 1

Introduction

1.1 Context Definition

Wildfires are large fires that quickly spread through dense forests and other areas. They have the power to destroy homes, animals, plants, and sometimes even humans. In addition, they emit a lot of gases and smoke, which decreases air quality and leads to climate change [2]. The incidence and severity of wildfires have recently increased. Human activity and climate change are the main factors that are responsible for this. Extreme weather, prolonged dry seasons, and higher temperatures contribute to the possibility of wildfires [3].

1.2 Motivation

There are many cases where wildfires cause severe damage. As reported by [1] in 2024, Sicily accounted for about 25% of all wildfires in Italy in that year. Approximately 2588 hectares of forest were burned on the island. Unfortunately, this is about 0.8% of the total forest cover in the area and 15% of all the land that burned in Sicily in 2024. Evergreen broadleaves (ECM-F4 T2) and conifers (T3) suffered the most damage. After adjusting for the relative abundance of each forest type, conifers (T3) suffered the

most, with a damage rate of more than 1.5%. The goal is to deliver a lightweight, real-time detection system designed to help responders act faster and preserve more forest. Figure 1.1 shows the Sicilian areas affected by wildfires and Table 1.1 shows a detailed breakdown by province.

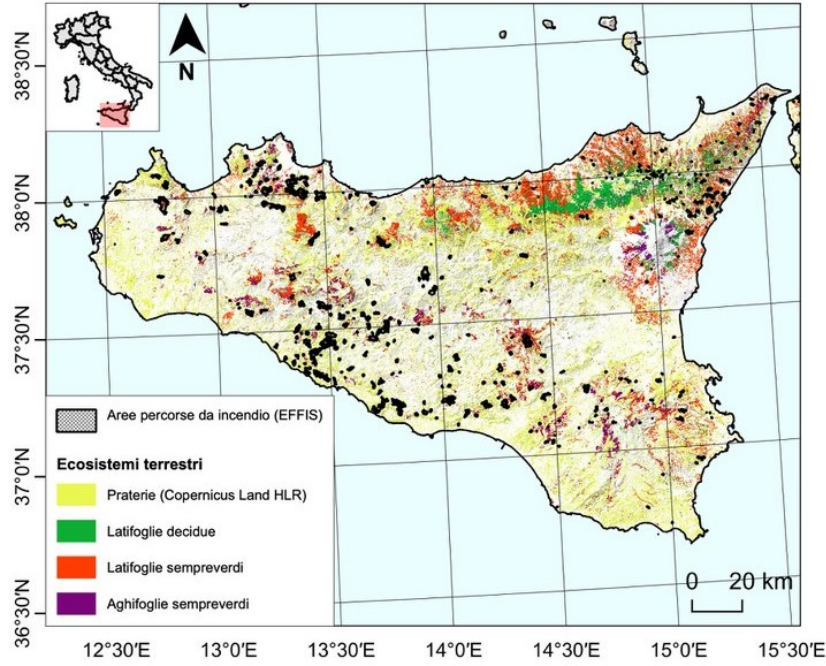


Figure 1.1: Burned areas in Sicily during 2024, Source: ISPRA [1].

Province	Unclassified forest	Deciduous broadleaves	Evergreen broadleaves	Evergreen needleleaf	Total burned
Enna	20	52	432	227	731
Catania	22	46	356	297	721
Messina	25	127	268	7	427
Caltanissetta	8	10	111	69	198
Agrigento	11	5	97	81	194
Palermo	5	5	89	5	104
Siracusa	8	8	85	2	103
Ragusa	19	6	40	34	99
Trapani	1	0	10	0	11

Table 1.1: Burned forest area by province in Sicily in 2024(hectares). Source: ISPRA[1].

1.3 Problem Identification

Today, there are many problems and risks associated with the actual wildfire suppression techniques. First, they respond frequently slowly and it can be much more difficult to detect and extinguish a fire that has spread over a large area by the time it is discovered and the firefighters arrive [2]. In addition, the cost of fighting fires with planes, helicopters, and firefighters on the ground is high because it involves resources such as fuel, equipment, and manpower [2]. Finally, these approaches are often not effective, especially in isolated or hazardous locations. Firefighters may find it difficult to arrive quickly in places with dense forests, rocky landscapes, and inaccessible locations. Due to this delay, the fire has more time to spread and is more difficult to control [2].

1.4 Proposed Solution

In the proposed system, once a fire is detected, a containment drone navigates to the scene and releases water or retardant from a safe standoff distance to support night containment (wet lines, hotspot knockdown). The UAV complements ground crews and daytime aviation by enabling operations after sunset, when helicopters and Canadair scooping aircraft are grounded for safety.

This approach seeks to be both efficient and rapid. Unlike traditional methods, drones could respond immediately, greatly reducing the time required to start fighting the fire. Due to their ability to be active first responders, drones could drop firefighting supplies precisely where they are needed, putting out fires before they get out of control. In general, this could significantly increase the speed, lower cost, and better quality of detection and suppression processes.

Consultation note. In line with guidance from the Protezione Civile della Regione Sardegna, this thesis proposes the use of UAVs to release suppressant from a safe standoff distance at night. The authority advised against impact/sacrificial use due to the risk of battery or fuel fires, and noted that, for safety reasons, helicopters and Canadair scooping aircraft do not fly after sunset, making UAV night containment the appropriate solution.

1.5 Objectives

There are a few main goals for this project. They are listed below:

Fully compliant architecture The thesis proposes an architecture that includes Wireless Sensors Network (WSN) and drones to detect and suppress the wildfires as quick as possible, reducing the risks for first responders and the costs for expensive resources (e.g., airplanes, helicopters)

Enhance fire detection method The thesis proposed supervised and unsupervised AI methods to make automatic the detection of wildfires through drones.

1.6 Feasibility and Limitations

For matter of available resources, this thesis does not cover a full implementation of all above-mentioned technologies. This thesis demonstrates the feasibility of AI fire detection methods, but does not include any implementation or deployment for WSN and drones. These parts of the architecture are included to build a full conceptual architecture and to provide advice for future works. The performance of the AI methods are relevant, while it will be important to test the foam drop in real or simulated fire conditions in the future.

1.7 Structure of the Thesis

This thesis is written as a short journey from the problem that is addressed to something that could work in the real world. In Chapter 1, the scene is explained: why wildfires matter, what problem is tackled, what is proposed, the goals that are set, and the limits caused by time and resources. In Chapter 2, what already exists is mapped, describing how fires are detected and fought today, why pairing WSNs with drones makes sense, and what the literature shows about suppression methods and costs so that the choices appear grounded. In Chapter 3, the approach is presented: the overall architecture

and assumptions, the dataset and preprocessing, and two AI paths (supervised and unsupervised), with the models, training setup, and a fair plan to evaluate them. In Chapter 4, the findings are shared, including metrics, examples, how tuning the decision threshold changes outcomes, and a small deployment simulation to illustrate how these choices would play out. In Chapter 5, the main lessons are summarized and the next steps are listed: moving WSNs and drones from concept to practice, testing foam drops in real or simulated fires, and improving the models with more data and better training. The usual lists of figures, tables, and code are also included so that details can be found quickly without breaking the flow.

Chapter 2

State of the Art

In order to help drones to react more quickly, different methods that are used today to detect forest fires will be checked and compared to determine which is more effective than the others. Fast and accurate fire detection is crucial because fires must be extinguished before they spread and become hard to control. Various fire-detection techniques will be compared based on accuracy (with few false alarms), cost, speed of detection, and performance in different scenarios. The objective is to identify the most effective technology to alert drones quickly so they can arrive at the fire and suppress it. After comparing these methods, the aim is to determine the best way to detect fires early and manage them more effectively.

2.1 Overview of Fire Detection Systems

2.1.1 Human Based Observation

This method is so basic, people stay on a tower and watch for fire. They look for smoke or flames to find a fire. Human based fire observation is cheap and easy to set up, it just requires people and some simple tools. But unfortunately it is unreliable because fires can be missed, especially in dense forests. It also has a slow response time and it always need many staff, making it hard to set up across large or difficult areas [2].



Figure 2.1: Example of a human based forest fire observation tower

2.1.2 Satellite Systems

Satellites in space have sensors that can take picture from the Earth. These sensors can notice heat or smoke and it gives them ability to detect fires across very large areas.

Two well-known satellites used for this purpose are AVHRR, launched in 1998, and MODIS, launched in 1999. They were designed to help with fire detection. The problem is that they can only take pictures of the same place on Earth about once in every two days and this is too slow and late if we want to catch fires early. Another big problem is that the images can be unclear when weather is bad like when it is too cloudy.

Even though there are so many downside and limits satellites are still helpful because they can watch over huge regions like places that are hard for people to reach (like mountains) and they can track how a fire spreads. But they also come with big downsides with the way we want to detect and suppress and does not align with our goal: they cost a lot of money to build and launch, they don't provide constant monitoring, and their view is often blocked by clouds or other weather problems [2].

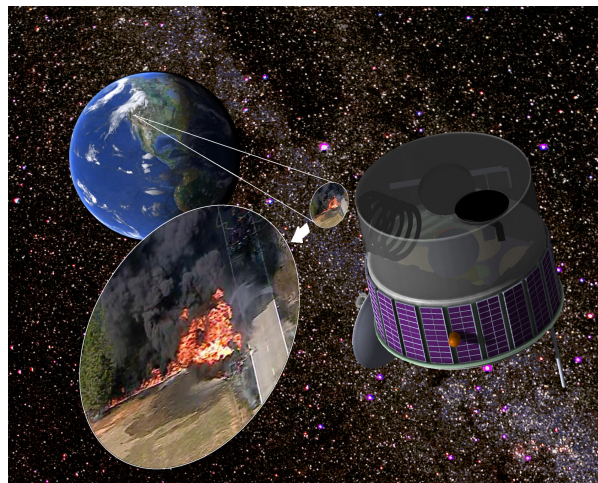


Figure 2.2: Example of a Satellite System capturing flames

2.1.3 Optical Surveillance Systems

Optical systems find fires by using light or heat signals to find smoke, flames, or hot spots. Thanks to improvements in technologies like better sensors, digital cameras, image processing, and faster computers, now there are systems that can automatically recognize signs of a fire and send out early warnings.

The best thing about optical systems is that they can keep an eye on big areas all the time and continuously and find fires early. Cameras take pictures to find smoke or flames and spectrometers and LIDAR look at light or laser signals instead of pictures. This makes it faster and more reliable to find things.

Different types of sensors are used for fire detection and each of them has its own way of noticing smoke, heat, or flames used for different purposed.

1.Video Cameras

These are just normal cameras which can capture visible fire signs, so during the day time they can spot smoke, and at night when smoke might not be visible they detect the flames.



Figure 2.3: Example of a Video Camera for forest fire observation

2. Infrared (IR) Cameras

These cameras detect the heat which is emitted by fire so it means in contrast to normal video cameras they can detect the fire even if flames and smoke are not visible

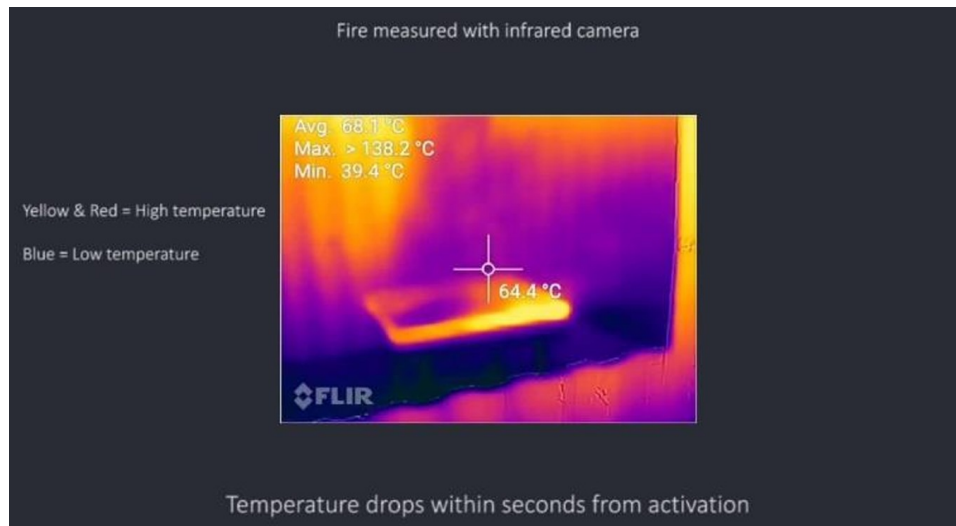


Figure 2.4: Example of an IR Camera for capturing flames

3.IR Spectrometers

These instruments work by looking at the infrared spectrum. Smoke particles and flames both change how light behaves, and they make a certain pattern that can be measured to find the fire. The spectrometer can tell when smoke or fire is present by looking at these changes. This is different from regular cameras and watchtowers. These tools can still find the fire even if you can't see it. This makes them so good for finding fires early especially when visibility is poor.



Figure 2.5: Example of a IR spectrometer for fire detection

4.LIDAR Systems

LIDAR(Light Detection and Ranging) Systems: LIDAR works by sending laser beams to the environment to scan it. When these beams hit smoke or fire particles part of the light return back to the sensor at this point the system measures how long it takes for the light to return from the environment and how the signal has changed and from this information LIDAR can confirm the presence of smoke and light with high accuracy. Because it can make these measurements very fast, LIDAR is very effective for real time fire monitoring and early warning.

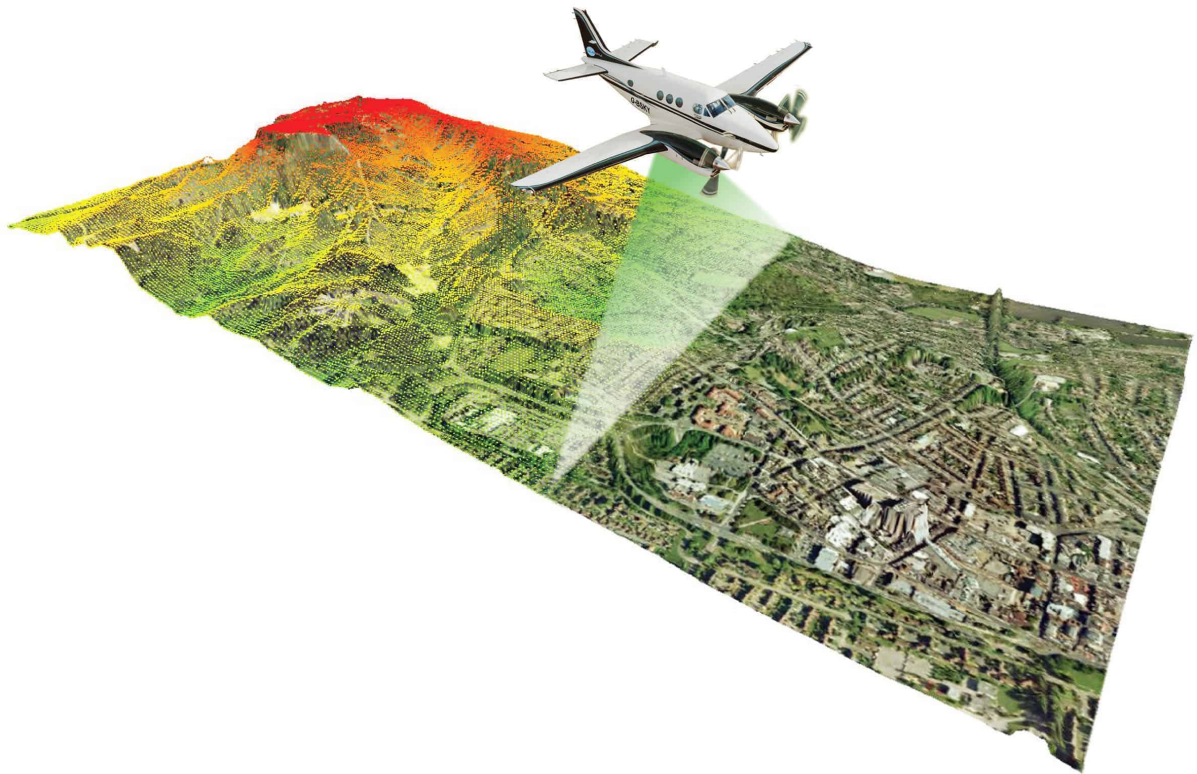


Figure 2.6: Example of a LIDAR for forest fire detection

Not all optical systems work the same way for example camera based systems can capture images over time and analyze the pixels of images to detect changes such as moving smoke or sudden flames. When enough warning signs are present then algorithms decide if to trigger an alarm or not and in many cases the system is linked with maps to show the fire's location. Other technologies, such as spectrometers and LIDAR, do not take pictures but instead study light signals or laser reflections to identify smoke or heat patterns in real time.

Cameras are more popular because they provide continuous monitoring and are generally more affordable than satellites but still they have limits like their view can be blocked by terrain or weather, they may confuse fog or reflections with smoke, and small fires often go unnoticed until they become larger [2].

2.1.4 Wireless Sensor Networks (WSNs)

Wireless sensor networks (WSNs) are becoming a common tool for detecting forest fires at early stage. They have many small sensors placed in the forest that measure things like temperature, humidity, and gases such as carbon monoxide and.... When they collect information, it is sent wirelessly to a base station for analysis. Each sensor runs on a small battery and combines sensing, processing, and communication on one tiny circuit board. Since they cannot be recharged regularly like a mobile phone so WSNs are designed to use very little power, and some are supported with small solar panels. A widely used technology in these networks is ZigBee, which is known for its low energy use and short range communication [2].

Now let's compare these systems to find the best option. Older fire detection methods like watchtowers, cameras, and satellites, were not always reliable in spotting fires fast especially during bad weather. But fortunately wireless sensor networks (WSNs) can provide real time data they can be organized into a stable and efficient network.

Table 2.1: Comparison of Fire Detection Systems

Feature	WSNs	Optical Surveillance Systems	Satellite Systems
Cost	Medium - initial setup and maintenance	Medium-High - RGB/thermal/spectral sensors, mounts & comms	Very High - satellite launch & maintenance
Detection Speed	Fast - real-time data	Fast - continuous streams with real-time processing	Slow - typically every 1-2 days
Accuracy in Early Detection	High - detects temperature, humidity changes	Medium-High - good with thermal/spectral; limited by line-of-sight	Low - can miss small or early fires due to coverage
Coverage Area	Localized, specific to sensor placement	Moderate - limited to field of view & terrain occlusion	Large - global coverage but not continuous
Reliability in All Weather	High - detects physical changes in environment	Medium - affected by fog/rain; thermal better at night/smoke	Low - cloud cover and weather distort images
Maintenance Needs	Moderate - battery replacement, repairs	High - lens cleaning, recalibration, alignment of optics	High - costly maintenance and updates
Power Source	Low power consumption; often solar-powered	Medium-High - continuous imaging & data transmission	Self-powered but requires significant energy
False Alarms	Low - measures actual changes in environment	Medium - fog/steam/sun-glint or heated surfaces can trigger	Low-Medium - may trigger from non-fire heat sources
Location Precision	High - accurate data from sensors	Medium-High - triangulation/calibrated mounts improve accuracy	Low - heat or smoke seen from afar, not precise
Ability to Self-Heal/Adapt	Yes - self-organizing network to reroute data	No - fixed viewpoints; coverage gaps if a unit fails	No - fixed orbit paths
Scalability	High - can add more sensors to expand coverage	Medium - adding units improves coverage but raises costs	Low - difficult to expand satellite coverage
Power Consumption	Low - designed for long-term battery life	Medium-High - continuous operation and transmission	High - significant power for long-distance transmission

2.2 Why WSNs and Drones Go Well Together

Different fire detection systems were analyzed, and Wireless Sensor Networks (WSNs) were identified as the most suitable approach for enabling drones to detect and suppress fires.. This is why:

2.2.1 Quick and Real-Time Detection:

WSNs can find fires as soon as they start because they can quickly pick up changes in temperature, humidity, and gases. This is very helpful for drones, which need to get accurate information quickly so they can act before the fire spreads.

2.2.2 Very Accurate for Early Detection:

These sensors are very good at understanding even small changes in the environment, which makes it easier to find fires early so the drones can go to the specified location very soon before it spreads and get big.

2.2.3 Exact Location:

WSNs tell you exactly where fires are happening. This is very important for drones because they need to know exactly where to go and where to drop their fire-fighting supplies so the supplies won't be wasted.

2.2.4 Works in All Weather:

Fog, rain, and darkness don't easily affect WSNs. This means that the information they send to drones is always accurate, no matter what time of day or night it is and we do not need to be worried to use another method in different seasons or different hours of the day.

2.2.5 Low Power and Self-Healing:

WSNs don't use much power, and they can often run on solar energy by using solar panel this makes them easy to maintain so they do not need so many personnel and expenses. The network can still work even if some sensors break or stop working. It does this by sending the data to a different place, which keeps the monitoring going.

2.2.6 Comparison with Other Systems

Cameras:

Obstacles and bad weather can block cameras and this can cause delays or false alarms but drones need data that is clear and trustworthy, but cameras might not always give this [2].

Satellites:

Satellites cover a lot of ground but they don't give you data in real time and we cannot put the fire out before it spreads. They only send pictures every one to two days, which is too slow for a quick drone response [2].

Conclusion

In conclusion, WSNs are the best way to help drones find fires. They give accurate data in any weather and in real time, and they show exactly where the fire is which makes them perfect for quick drone action to put out fires.

2.3 Comparison of WSN-Based Fire Detection Systems:

Several projects that have used Wireless Sensor Networks (WSNs) for forest fire detection are reviewed here:

2.3.1 Lloret et al. (Spain):

Developed a system that combines a mesh network of sensors with IP cameras to detect fires early. When sensors notice signs of a fire, nearby cameras are switched on to confirm it. The problem with these type is that sending images uses a lot of power and the cameras only perform well in clear weather [2].

2.3.2 Son et al. (South Korea):

It is a fire detection approach that uses wireless sensor networks to track temperature and humidity, with data sent to a central base station. But the downside is that instead of directly detecting fires its main purpose is to evaluate fire risk levels based on weather conditions so the focus is more on risk assessment than on the reliability of the network itself.

2.3.3 Hartung et al. (FireWxNet):

Which monitors forest weather conditions with a mix of wireless sensor nodes and web cameras. But again it is mainly used to record weather conditions and study how fires behave once they start. However, it is not designed as a tool for early fire detection [2].

2.3.4 Alonso et al. (ZigBee Network for Fire Detection):

It is made from of low power sensors and these sensors can track temperature, humidity, and gas levels, which are important signs of possible fires. ZigBee has some protocols that makes its energy consumption so low and this allows sensors to run on batteries or small solar panels for long periods. The system sends real-time data to a base station and gives fast alerts for early detection and this is very important in my drone based approach. Its design emphasizes both efficiency and reliability, making it practical for continuous use in remote forests [2].

Conclusion

In summary, wireless sensor networks are becoming an effective tool for detecting forest fires. They are more efficient than older methods and can deliver real-time alerts, which helps speed up the response and reduce fire damage.

2.3.5 Which WSN to use?

Out of the four systems reviewed Alonso et al.'s ZigBee based fire detection network stands out because it is the most efficient and practical for early fire detection. The main reason is its use of ZigBee technology which is designed for low power use, real-time data transfer and fast alerts.

By comparison, Lloret et al. (Spain) depends on IP cameras to confirm fires but this method uses a lot of power and often fails in bad weather. Son et al. (South Korea) focuses on measuring temperature and humidity but the goal is more about assessing fire risk than giving quick alerts and the wireless communication seems less efficient. Hartung et al. (FireWxNet) collects useful information on weather and fire behaviour but yet its reliance on web cameras brings similar issues with power use and reliability.

In contrast, Alonso et al.'s system is directly optimized for early detection. By monitoring key signs like temperature, humidity and gas levels it can quickly send alerts while remaining energy efficient and reliable even in remote forest areas. In conclusion Alonso's approach is the best suited for fast fire detection compared to the others.

2.3.6 ZigBee architecture

This image shows a ZigBee-based wireless sensor network used for wildfire detection. Small and low power consuming sensors are placed around the forest and linked together in a mesh network where each sensor passes information to the next until it reaches monitoring center. This setup lets the system to understand changes in temperature, smoke, or other signals quickly so fires can be detected at early stage. Because of the mesh design the network can keep working because if one sensor fails other sensors can reroute the data and send the data through other nodes instead. These sensors cover wide areas and give real time alerts and improve the response time and can also track other conditions like weather or wildlife activity. This approach is efficient and often powered by solar panels but unfortunately it still needs some initial setup and regular maintenance to keep all sensors charged and working properly.

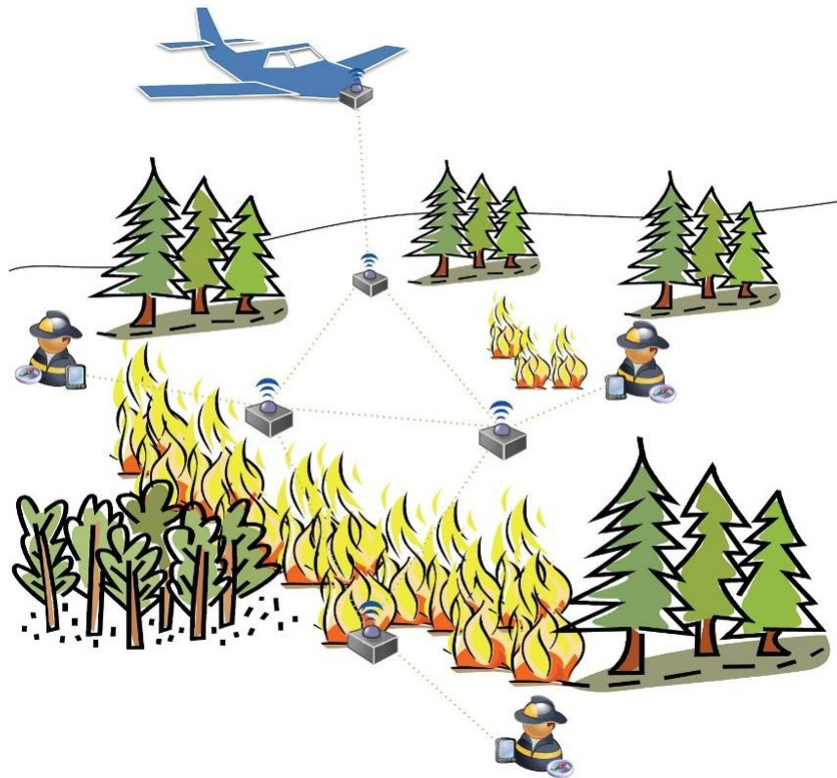


Figure 2.7: Example of ZigBee based wireless sensor network mesh for forest fire detection

2.4 Principles and Methods of Fire Suppression

After concluding that Wireless Sensor Networks (WSNs) are the best for detecting fires, next step is to find the best way to put those fires out. Fires need three things to burn: oxygen, heat, and fuel (like wood or leaves). This is called the fire triangle and the goal is to break this triangle by removing at least one of these elements [4].

2.4.1 Understanding and Breaking the Fire Triangle

- **Oxygen:** Fires need air to burn and since the atmosphere has plenty of it it's usually not easy to remove it specially in large fires which is so hard to cut off the oxygen. But doing this can stop a fire.
- **Heat:** Heat is what keeps the fire continuing. Taking away the heat, like cooling it with water, stops the fire.
- **Fuel:** This is anything that can burn. If fuel (like plants, trees or....) does not exist then the fire has nothing to burn.

These three elements are all essential for a fire to survive and continue. If any one of them is removed even partially the fire will be weaker and eventually finish [4], [3]. Fire suppression techniques are built around this principle, combining different methods to contain, control, and extinguish wildfires as efficiently as possible.

2.4.2 How Fire Suppression Works

Remove Fuel:

Clearing dry plants or other flammable material creates barriers that stop the fire from spreading.

Cut Off Oxygen:

Smothering small fires with dirt, blankets or similar materials does not let the flames to have oxygen.

Cool the Heat:

Spraying water or fire retardant chemicals lowers the temperature enough to extinguish the fire.

Firefighting methods focus on breaking one or more sides of the fire triangle to bring the flames under control and prevent further damage [4], [3].

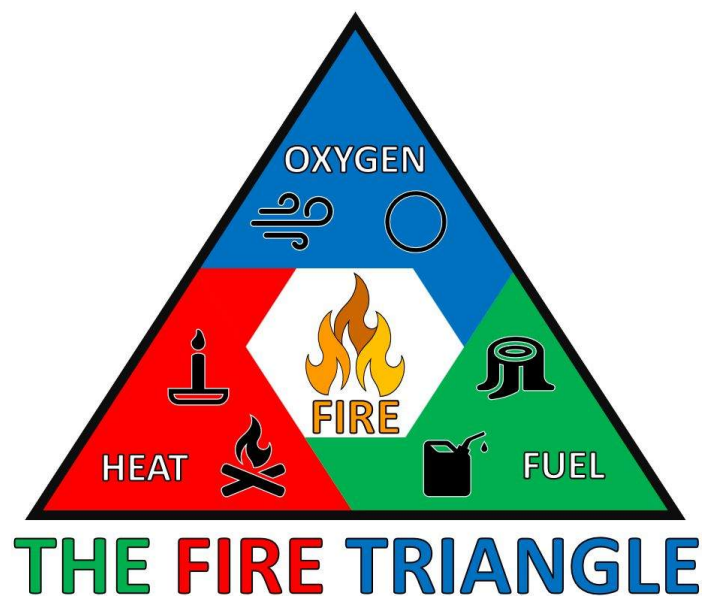


Figure 2.8: Three fire triangle elements

2.5 Fire Suppression Techniques Overview

There are several ways to control wildland fires, including direct attack, indirect attack, flank attack, and parallel attack. Each method has its advantages and disadvantages depending on the size, behaviour, and intensity of the fire. Below is an overview of each approach, including its pros and cons:

2.5.1 Direct Attack

Firefighters fight fires directly by working right along the edge of the fire with hoses, shovels, and fire-retardant materials to either put out the flames or slow their spread. When flames are less than four feet high this method works best because it lets firefighters to get close safely and not getting harmed. When flames are between four and eight feet high, it may be necessary to use heavier equipment like bulldozers or drops of water and retardant from planes to stop the spread. Direct attack is best for small fires or less intense parts of bigger fires because it lets you respond fast and precise. Firefighters can put out flames before they get bigger and have more control over where the fire stops. But this method is not safe because crews have to work close to the fire line, which can be dangerous if things change quickly. It is also not practical for big, fast moving fires, especially when the flames are more than eight feet high [4], [3].

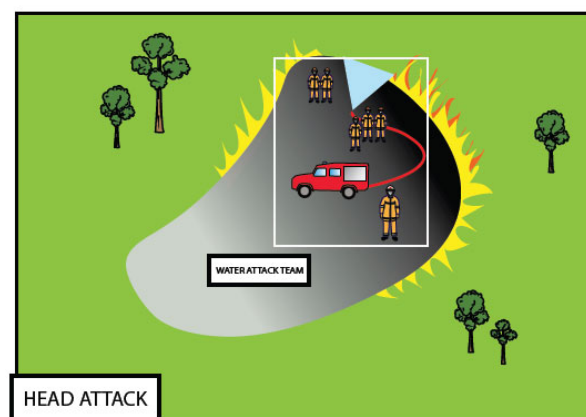


Figure 2.9: Example of Direct attack

2.5.2 Indirect Attack

This method involves putting up control lines (barriers) far enough away from the fire to be safe. Firefighters then create fire to the plants between the fire and the control line, making a buffer. To keep the fire from spreading. When the flames are more than 8 feet high or it's too dangerous to get close to the fire, this is used. An indirect fire attack safety is the first priority by keeping firefighters away from the flames. It also works well for big fires where it would be too risky to get too close to the fire. But it takes time to set up because control lines have to be made and plants have to be cleared or burned. This method can also hurt plants or buildings, since some land may need to be burned to stop the fire from spreading [4].

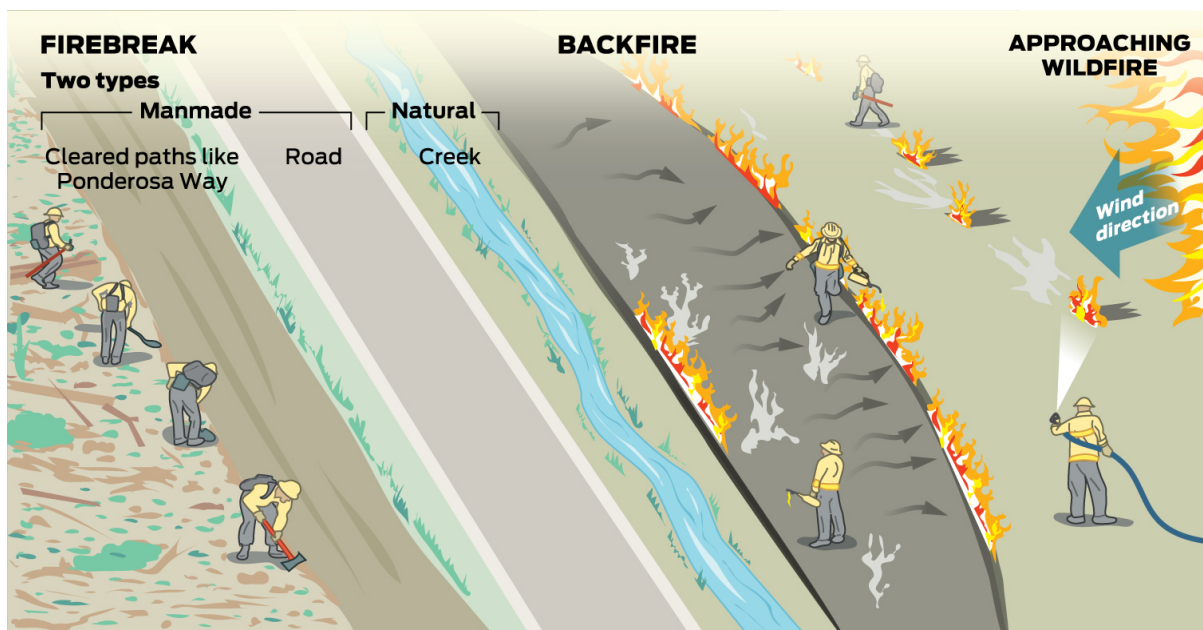


Figure 2.10: Example of Indirect Attack

2.5.3 Flank Attack

Firefighters do a flank attack by starting from a safe place (an anchor point) on one or both sides (flanks) of the fire and working their way to the head, which is the part that moves the fastest. The goal is to hold the fire back from the sides as it moves forward. It works well when fires are spreading at a normal speed. A flank fire attack helps keep the fire from spreading by stopping it from moving from the sides. This works well for moderate fires when the flames do not spread too quickly. But it needs to be done in a safe way, since burning out unburned areas between the fire and the control line can be dangerous. Also, this method might not put out the fire very quickly because it is usually slower than a direct attack [4].

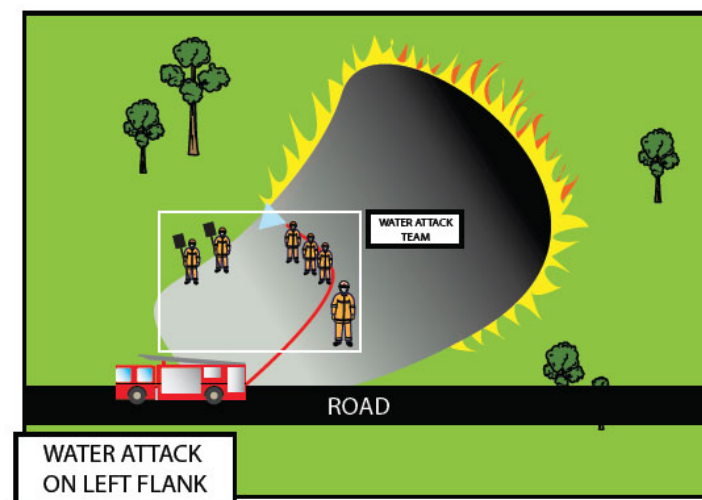


Figure 2.11: Example of Flank Attack

2.5.4 Parallel Attack

This method is like an indirect attack, but the control line is built much closer to the fire's edge, usually within 100 feet. Firefighters use machines to quickly build this line. When a direct attack isn't possible, but it's safe to work close to the flames, this is what you use. A parallel fire attack makes it easier to contain the fire faster because

the control line is closer to the fire than in an indirect attack. This method also saves more land because it burns a smaller area. But it needs to be close to the fire which means firefighters have to work closer to the flames and it is dangerous. This method also requires a lot of work and equipment because it needs machines and skilled workers to quickly set up effective control lines [4].

2.5.5 Conclusion

All of these methods want to stop a fire by breaking its triangle with removing at least one of the fuel, heat, or oxygen it needs to keep burning. The right method depends on the size of the fire, how intense it is and the target landscape. Direct attack works well for smaller and less intense fires because it can put them out very fast and save more land but it can be risky for firefighters and isn't always possible for bigger and fires. Reducing risks to firefighters and making direct attacks on larger fires possible would be the best approach. It would allow fires to be put out quickly at their source with fewer dangers to people. Using drones could be the key to making this happen. They can put out fires directly without putting firefighters life in danger.

2.5.6 Why Night Containment with UAVs Is Useful

Direct attack means working right at the fire edge to stop spread. That's risky for people and, after sunset, crewed aircraft are grounded for safety. A containment drone fills this gap: it flies to the perimeter and releases water or retardant from a safe distance. It does not crash into the flames. Instead, it lays wet lines, cools hotspots, and slows the fire so crews can take over in daylight.

Because it flies, it can reach steep, rocky, or roadless areas. Its drops are precise and low-volume, which helps where a targeted spray is better than a wide, heavy drop. By working just off the flame front—where visibility and heat allow—the drone not to put people in danger. Combined with WSN detection and the onboard AI that cues exact locations, the drone adds a night-time containment window that we do not have today.

2.6 Comparing night containment drones with helicopters and airtankers

This section compares **reusable containment drones** operating **after sunset** with traditional daytime aerial assets (helicopters, airtankers, and other fixed-wing aircraft). The roles are complementary: crewed aircraft deliver large volumes during daylight, while small UAVs provide **precise, low-volume, safe-distance drops at night** to wet control lines and cool hotspots. To ground the comparison, official wildfire aviation spending and the split between availability and flight-time costs is first summarized to establish a fair baseline for mission costing.

2.6.1 Cost context

According to the USDA Forest Service 2022 Aviation Annual Report, contract aircraft expenditures totaled \$789.7 M. By aircraft type, the cost share was Helicopter 59.1% (\$466.6 M), Airtanker 21.6% (\$170.7 M), Fixed-wing 9.8% (\$77.1 M), and Scooper 9.5% (\$75.3 M). Costs by pay code show a strong availability component (63.9% of total) while flight time accounts for 31.1% (standby/other). In practice, even short sorties incur substantial fixed costs from availability-an important consideration when comparing with night UAV operations, which do not incur standby fees. [5].

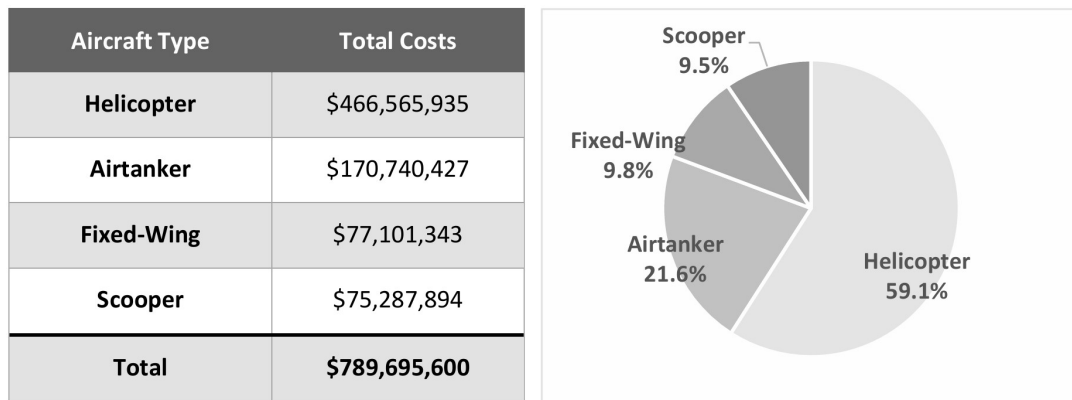


Figure 2.12: Contract aircraft expenditures according to the USDA Forest Service 2022 Aviation Annual Report.

This table provides the two inputs used in the aircraft mission-cost model below-hourly flight rate (\$/hr) and fuel burn (gph)—for three representative helicopter classes (Type 3/2/1). Jet-A price is treated as a parameter; where an example is needed, the rate card’s reference value \$5.19/gal is used.

Table 2.2: Representative helicopter inputs for cost modeling.

Model	USFS Type	Fuel (gph)	Hourly flight rate (\$/hr)
Airbus AS-350B2	Type 3 (light)	48	1,219.00
Sikorsky S-70 / UH-60	Type 1–2 class	160	4,280.03
Boeing CH-47 (BV-234)	Type 1 (heavy)	405	7,724.87

Source: U.S. Department of Agriculture [5].

The next summary shows five years of helicopter activity and how total spending splits between availability and flight/other costs—explaining why short flights can be expensive (high availability share).

Table 2.3: Contract Helicopter Use and Costs Summary.

Year	Flight Hours	Availability (M USD)	Flight & Other (M USD)	Total (M USD)
2018	39,892	\$198.0	\$141.0	\$339.0
2019	20,588	\$145.2	\$64.3	\$209.5
2020	42,667	\$251.0	\$151.4	\$402.4
2021	43,272	\$265.5	\$184.3	\$449.9
2022	30,072	\$319.5	\$147.1	\$466.6
<i>5-yr avg.</i>	35,298	\$235.8	\$137.6	\$373.5

Source: USDA Forest Service, 2022 Aviation Annual Report [5].

This table anchors the fixed-wing airtanker side: representative models, their flight hours, retardant delivered, and availability costs. It highlights the high throughput of LAT/VLAT (large and very large air tanker) platforms and their availability-heavy cost structure.

Table 2.4: 2022 Contract LAT/VLAT Use Summary by Aircraft Model

Aircraft Model	Flight Hours	Retardant (gallons)	Availability Costs (USD)
BAe-146	1,756.6	4,934,307	\$26,008,122
RJ85	1,706.6	3,812,210	\$31,369,343
DC-10	1,139.0	7,162,235	\$21,752,512
MD-87	1,189.2	3,607,265	\$20,569,146
EC-130Q	252.4	725,465	\$4,027,107
B-737	330.8	1,063,335	\$9,819,500
Total	6,374.5	21,304,817	\$113,545,730

Source: USDA Forest Service [5].

This table provides conservative acquisition anchors for small COTS UAVs that can be adapted as **reusable containment drones**. These anchors are used later to parameterize per-mission cost via amortization (no availability charges).

Table 2.5: COTS UAV acquisition anchors (< USD 5,000) and flight time.

Model	Price (USD)	Flight time (min)
DJI Mavic 2 Pro	1,599.00	29
DJI Mavic 2 Zoom	1,349.00	29
DJI Mavic Air	919.00	20
DJI Mavic Pro Platinum	1,149.00	30
DJI Phantom 4 Pro V2.0	1,599.00	30
DJI Inspire 2	3,299.00	23–27
Parrot ANAFI Work	999.00	25
Parrot ANAFI Thermal	1,900.00	26
Parrot Bluegrass Fields	4,980.00	25
Yuneec Mantis G	699.99	33
Yuneec Mantis Q	499.99	33
Yuneec Typhoon H	899.99	25
Walkera Vitus	739.00	28
Walkera Vitus Starlight	899.00	22
HolyStone HS720	299.99	26
HolyStone HS120D	159.99	16
HolyStone HS100	169.99	12–15

Source: Commercial UAV platforms [6].

Mission cost comparison: helicopters, airtankers (day) and containment drones (night)

Helicopter and airtanker mission cost.

The marginal flight cost is modeled as

$$\text{Cost} = h \times (\text{rate } [\$/\text{hr}]) + h \times (\text{fuel burn } [\text{gph}]) \times (\text{Jet-A } [\$/\text{gal}]),$$

where h is flight duration (hr), with Jet-A valued at \$5.19/gal per the USFS rate card [7]. In real contracts, **availability** dominates total expenditures, so short sorties can be disproportionately expensive [5].

Containment drone operating cost.

For a reusable UAV operated at night, the per-mission cost is parameterized as

$$\text{Cost}_{\text{UAV}} = \frac{\text{Capex}}{M} + c_{\text{consum}} + c_{\text{energy}} + c_{\text{maint}},$$

where Capex is purchase price, M is the amortized service life (missions), and c_{consum} , c_{energy} , c_{maint} are consumables (foam/water), battery charge, and minor maintenance. Unlike crewed assets, there is **no availability fee** for simply being on standby at night.

Worked example (no table). For a concrete number, a small UAV costs \$2,500 is supposed and is used for $M = 100$ missions. The amortized purchase is \$25 per mission. If consumables (foam/water and any liners) are about \$10, battery energy \$1, and minor maintenance \$6, then

$$\text{Cost}_{\text{UAV}} = 25 + 10 + 1 + 6 = \$42 \text{ per mission.}$$

If the UAV is cheaper and lasts longer (e.g., \$1,000 over 200 missions), the total can be about \$15 per mission; if it is more expensive and used fewer times (e.g., \$5,000 over 50 missions), it can be about \$125 per mission. These figures exclude labor/command overhead and, unlike crewed aircraft, there are no availability/standby charges for simply being ready at night.

Note. With this framing, the *price of the drone is included* through Capex/M , so the UAV numbers above are directly comparable to the *marginal* per-sortie costs shown for helicopters and airtankers.

How many drones are required? (night containment)

The number of UAVs depends on effective delivered volume along a control line:

$$N = \left\lceil \frac{V_{\text{required}}}{V_{\text{payload}} \cdot \text{expansion} \cdot \alpha} \right\rceil,$$

where V_{payload} is the liquid per drop, “expansion” is foam aeration, and α captures delivery efficiency for **safe-distance, off-edge** spraying (wet lines and hotspot cooling rather than entering the flame front).

Summary.

Daylight aviation is optimized for high-throughput delivery but carries substantial availability costs and daylight/visibility constraints. **Containment drones** add a night-time window with precise, low-volume, safe-distance drops and no availability charges. Together they are complementary: aircraft push large volumes by day; UAVs stabilize edges and hotspots at night so morning resources begin from a safer, more controlled baseline.

2.6.2 Conclusion (cost and beyond)

From the models above, the per-mission cost of small, reusable containment drones is modest and not driven by availability fees. Cost, however, is only one axis. Operational value also depends on delivery precision, launch latency, the safe operating envelope, and the risk imposed on crews. The points below summarize why containment drones are a useful addition for night operations and hard-to-reach ignitions.

- **Night operational window.** After sunset—when helicopters and amphibious water-scooping airtankers are grounded for safety—containment drones can create wet lines and cool hotspots from a safe standoff distance.
- **Precision at standoff.** Low-volume, close-range spraying enables targeted application along control lines and hotspots, reducing drift relative to higher-altitude drops when conditions permit.
- **Access to complex terrain.** Small UAVs can operate in steep, roadless, or canopy-constrained areas where positioning crews or establishing landing zones is difficult.

- **Rapid launch and distributed basing.** Light logistics allow staging near risk areas and launches within minutes, improving initial containment timelines.
- **Crew risk reduction.** Tasks near heat and smoke can be performed without placing pilots or firefighters in immediate danger.
- **Efficient suppressant use.** Targeted, low-volume delivery focuses on the highest-leverage locations (e.g., line breaks, hotspots), potentially reducing waste compared to broad-area coverage.
- **Sensor- and model-guided execution.** Integration with WSN alerts and on-board perception supports cue-to-target drops and post-drop assessment.

Limitations and fit. Small UAV payloads constrain throughput; performance depends on wind, smoke, visibility, and endurance; and operations must comply with airspace rules and incident coordination. Consequently, containment drones should be viewed as a *complement* to daytime aircraft and ground crews: they stabilize edges and hotspots overnight so morning resources begin from a safer, more controlled baseline.

Summary. The thesis therefore adopts **containment drones for night operations**—extending coverage into hours when crewed aviation cannot fly—using precise, safe-distance drops at modest per-mission cost, rather than attempting to replace high-throughput daytime assets.

2.7 Selecting the Optimal Foam for Drone Fire Suppression

In this section different firefighting foams are compared to find the best option for drone based fire suppression and the goal is to find a foam that can be deployed quickly and effective in extinguishing fires and it be safe for the environment by considering the all needs of drone to operate good.

2.7.1 Synthetic Foams (S1, S2, S3)

Synthetic foams are made using chemicals called surfactants which help the foam to expand and spread over a fire. These foams come in different percentages (3% or 6%) and it affects how well they work. They can be low, medium and high expansion, meaning they spread quickly and cover large areas in the forest. But their effectiveness can be different depending on their percentage and the type of fire they are putting out [8].

2.7.2 Aqueous Film-Forming Foams (AFFF)

AFFF foams are made to quickly cover flammable materials with a thin layer, cutting off oxygen and putting out the fire quickly. The regular surfactants and fluorosurfactants in them work together to make a barrier that protects the flames. They work well with seawater, cover a lot of ground, and are great for quick action. They grow slowly, so they can spread over fires quickly, but they might run out of water in 10 to 20 minutes [8].

2.7.3 Fluoroprotein Foams (FP1, FP2, FP3, FP4, FP5 AR)

Fluoroprotein foams are made from natural proteins with added chemicals that help them spread more easily over a fire. They form a strong layer that clings to the burning surface and prevents the fire from starting again, which makes them very stable. With an expansion ratio greater than 7, they can cover a large area and remain effective for a long time. Some types can even be mixed with seawater, making them useful in a variety of firefighting situations [8].

2.7.4 Film-Forming Fluoroprotein Foams (FFFP)

FFFP foams are made from protein based foams and other things that help them spread quickly and make a strong barrier over a fire. Their expansion ratio is a little lower (greater than 6), and they take a long time to drain (over 9 minutes) and this makes them good for putting out fires that last a long time. These foams are helpful when you need quick coverage and a long-lasting effect to keep the fire from starting again [8].

2.7.5 Protein-Based Foams (P)

Protein foams are made from natural protein sources so they are decomposable and vanish after some times and more environmentally friendly than other synthetic options we saw. They form a thick and stable surface over a fire and have a high expansion ratio (over 7). When used at a 6% concentration they provide long lasting coverage and are effective for firefighting operations. Because of their stability and environmentally friendly makeup, protein based foams are a good choice for controlling large fires while reducing environmental impact [8].

Table 2.6: Firefighting Foam Parameters

Firefighting agent	Viscosity at 20°C (mPa·s)	Biodegradation (%/28 days)	Compatibility with sea water	Expansion ratio	50% Drain time (min)
S1 3%	16±2	>90	–	>8	>15
S2 6%	4±1	88	–	>8	>15
S3 6%	10±2	NA	–	>8	>15
AFFF AR 3%	NA	NA	+	>7	10–20
FP1 6%	6±1	NA	–	>7	>15
FP2 6%	4±1	NA	–	>7	>10
FP3 6%	20±2	NA	+	>7	>9
FP4 6%	6±2	NA	+	>7	>9
FP5 AR 3%	>200	>92	–	>7	>12
FFFP AR 6%	<850	NA	–	>6	>9
P 6%	<25	NA	+	>7	>9

2.7.6 Conclusion

A synthetic foam that doesn't contain fluorine (S2, 6%) is chosen for night containment with UAVs that can be used again. The UAV releases a suppressant from a safe distance to make wet lines and cool hotspots. The agent must be able to be pumped through small hardware, spread well from small tanks, and stay on fuels. S2 has a low viscosity (it works with small pumps), a high expansion (it covers a lot of surface area), and enough drain time (it stays on the film), which makes it more effective with small payloads. Not having fluorinated compounds also helps the environment. For hydrocarbon spills near roads or airfields, incident-command policy may allow for the use of film-forming foams (AFFF/FFFP). For vegetation drops, S2 is still the best option.[8],[9],[10] .

Chapter 3

Material and Methods

Wildfires can change quickly. The wind may change direction, the flames may spread quickly, and by the time a drone gets there the scenario may be different. WSNs can help find the general area, but AI lets the drone look at the big picture from the sky in near real-time and pick the best spot to drop supplies for fighting fires. When drones are involved in the suppression of wildfires, it is very important to be accurate because drones can only carry a small amount of foam or water, and every drop needs to be used wisely. AI also improves navigation because it can help drones avoid things like trees, uneven ground, and smoke that WSNs cannot always see.

In short, WSNs help the drone find the general area of the wildfire, and AI makes sure the drone works right by hitting the right spots and adjusting to current conditions. Together, they make fighting fires faster, smarter, and more effective.

3.1 Architecture

The system collects data from a WSN, which consists of nodes spread throughout the target forest. These sensors monitor, for example, temperature, humidity and gas levels all the time to look for signs of a fire. When they notice something anomalous, data is sent from one node to another until it reaches the fire control station. Therein the information is processed to find out exactly where the fire is and how bad it is. Moreover, it sends

an alert to the drone launch station. At this point, a drone is sent to the scene of the fire and the AI onboard perception implemented detects and suppresses the fire. In fact, it can recognize the wildfire area and start fighting the fire by spreading fire retardant materials and starts fire suppression by releasing suppressant from a safe distance in support of night containment. This system is designed to be fast and efficient, so that fires can be found, put out, and stopped as quickly as possible to limit damage and stop the fire from spreading. Figure 3.1 shows the architecture design. Figure 3.2 reports the workflow of the components in the designed architecture. Figure 3.3 describes how the component interact with each other.

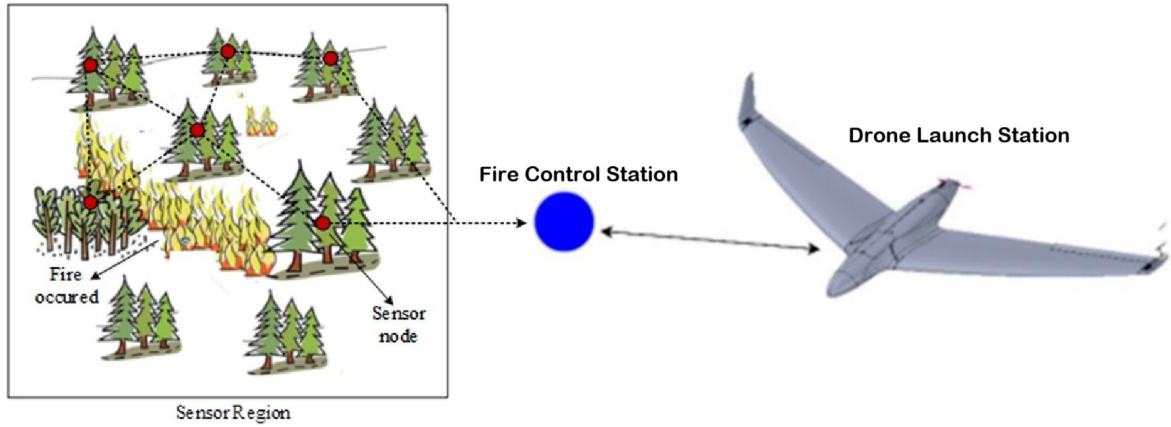


Figure 3.1: Proposed architecture.

3.1.1 Assumptions

This thesis assumes that a WSN and a base station are already deployed in a forest. It can monitor temperature, humidity, smoke and gas in the area and sends the information to the base station. The base station gets these readings, checks them, and then decides when and where a drone should take off. This thesis covers the part of the drone that lets it look at pictures and decide what to do next (both a supervised and an unsupervised version).

simplified activity diagram of how drones work using WSNs

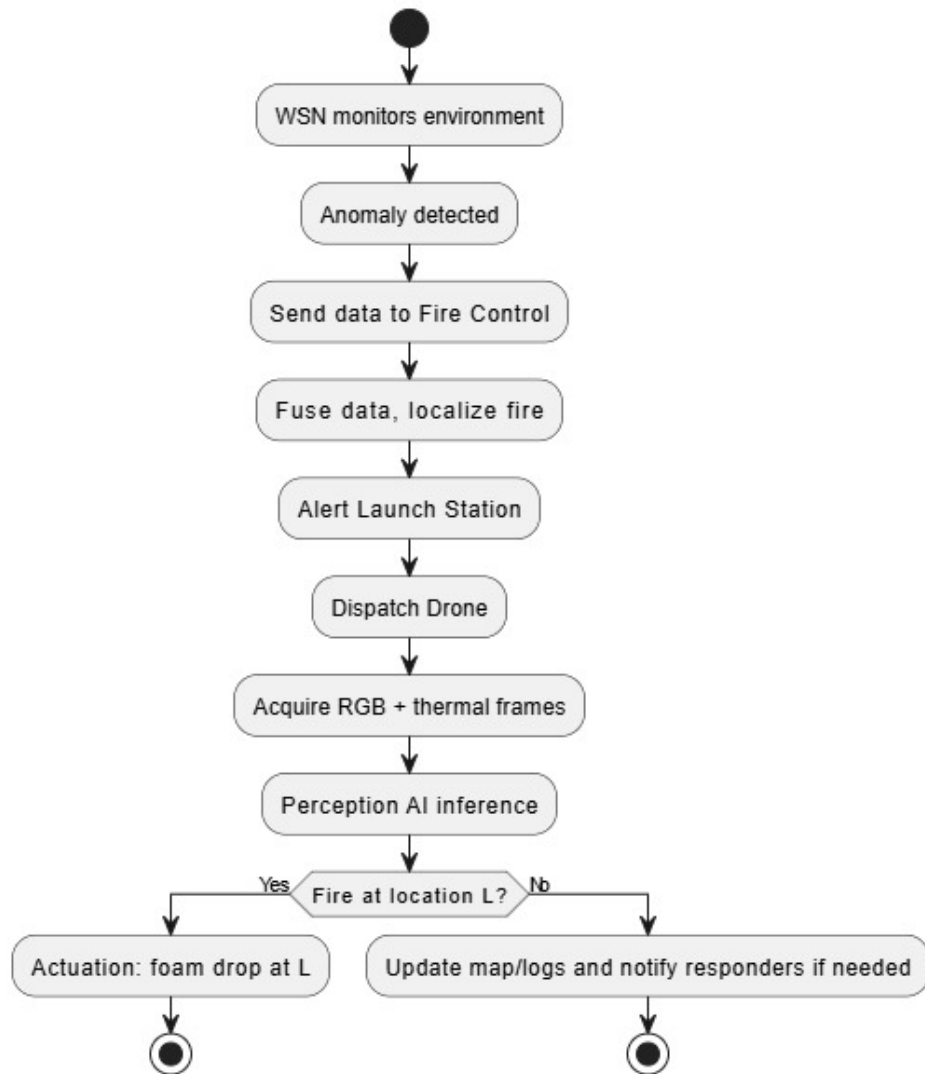


Figure 3.2: Simplified activity diagram of how drones work using WSNs

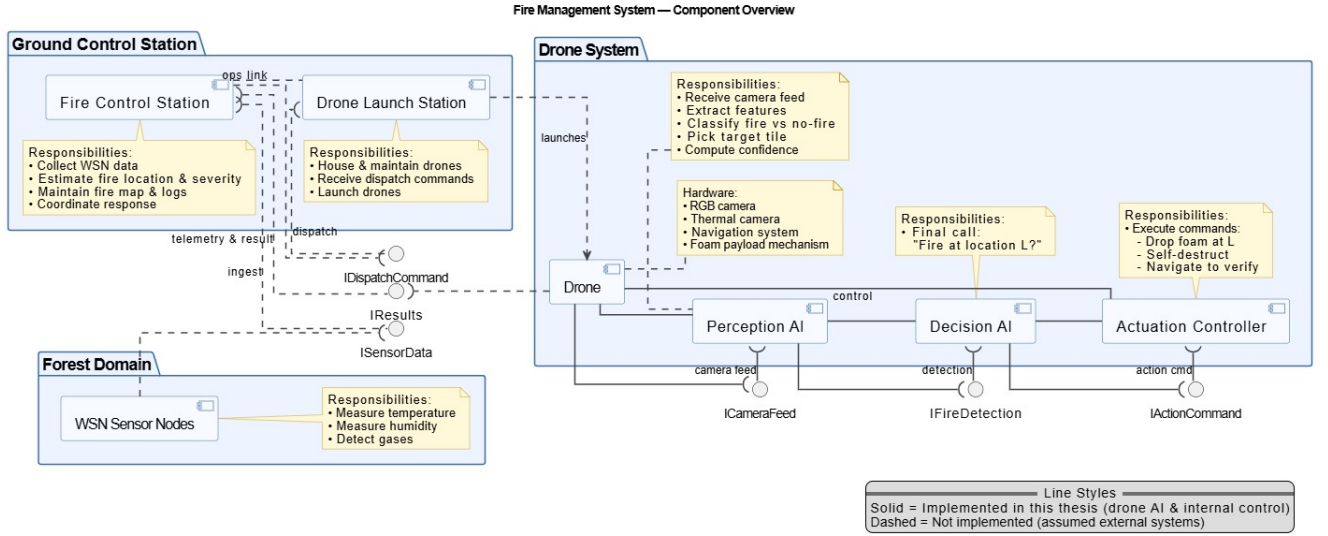


Figure 3.3: Component diagram of how drones work using WSNs

3.1.2 Proposed Components

The drone component has two image pipelines: i) a supervised model that gives a probability between 0 and 1; and ii) an unsupervised model that gives a risk as classes 0 or 1. The two methods cover different needs.

The supervised approach is good when the scene looks like examples it has seen before, and it gives a clear probability the base station can sort and rank. On the other hand, the unsupervised approach does not learn from labels. It looks for what is unusual in the current mission, so it can flag new or odd cases (e.g., different angles, smoke color, lighting, fog, backgrounds).

This setup is practical in the field. The unsupervised approach works on day one when deployed in a new area, while the supervised approach may improve slowly as many pictures are collected and labeled. The methods may be used at the same time to maximize the capacity to detect a wildfire. This means that an ensemble may be used to merge the two inferences.

3.2 Dataset

The dataset adopted for this thesis has images with and without fire labels. The pictures represent forests and other outdoor areas. They show flames, distant smoke, sun glare, fog, and other things that can look like fire. As shown in Figure 3.4, there are two main parts to the directory: `train/` and `test/`. There are two subfolders in `train/`: `Fire/` and `NoFire/`. Each folder has the images for the class. The `test/` part is a flat folder, so it does not have any class subdirectories, and it is mixed. The label is still in every test image, but is in the filename (it has either "fire" or "nofire" in it), so the ground truth is visible.

```
dataset/  
|- train/  
|  |- Fire/  
|  `-- NoFire/  
`-- test/  
    |- 01234_fire.jpg  
    |- 04567_nofire.jpg  
    |- 07890_fire.jpg  
    `-- ...
```

Figure 3.4: Dataset layout. `train/` is labeled via subfolders and filenames. `test/` is flat with labels in filenames.

It depends on the pipeline how these labels are used. The `train/Fire/` and `train/NoFire/` folders are kept as they are in the supervised pipeline. Both folder names (and the labeled test filenames) are used directly as ground truth. When features and cluster images are built in the unsupervised pipeline, the system does not pay attention to the labels. Instead, train images are mixed into a single pool so that the clustering does not know what the classes are. Only after clustering, the system looks at the labels. First, it use the train labels to figure out which cluster is mostly Fire and which is mostly No Fire. Then, the label is used in the test filenames to check the final results. In this way, dataset stays the same for both methods, so the final comparison will be fair. The only difference is whether it use the labels to learn (supervised) or just at the end to name and evaluate (unsupervised). Figure 3.7 shows sample of the dataset.

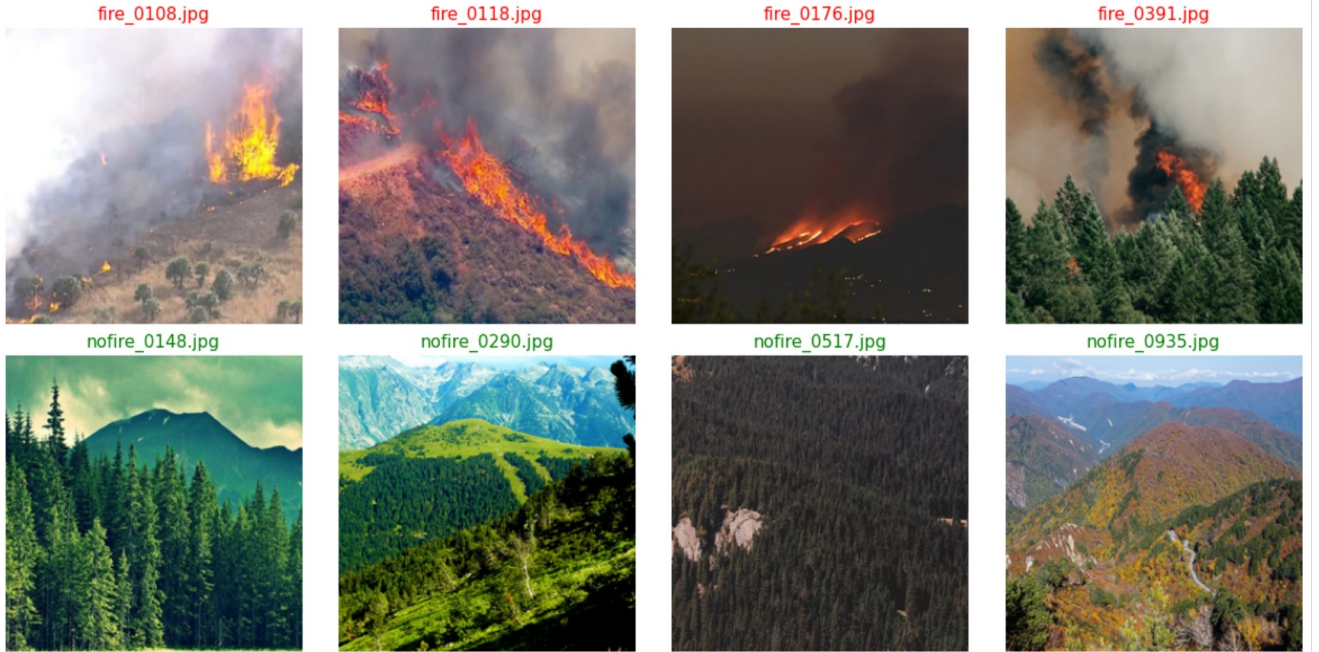


Figure 3.5: Dataset sample images

3.3 Pipelines

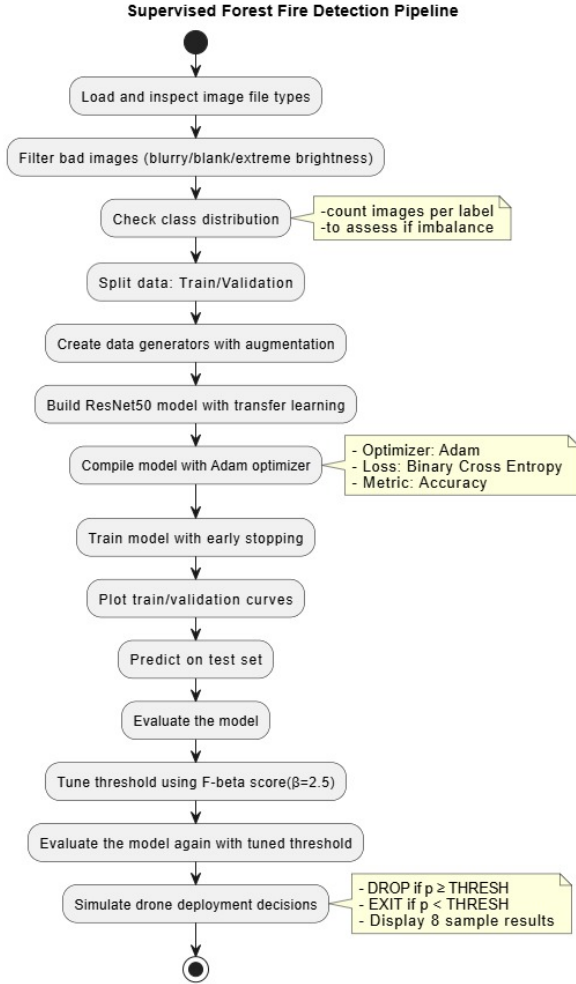
Two AI pipelines have been built, so they are comparable and fair. The goal is to judge a supervised and an unsupervised approach on the same ground, with the same images and the same basic rules. First, dataset is cleaned the same way for both: it drops blurry, blank, or extreme brightness files. Then it maintains one consistent preprocessing path with a shared backbone (ResNet50). In the supervised path, it adds a small classification head and trains it with early stopping. In the unsupervised path, it leaves the backbone headless and only extract embeddings without labels used.

To avoid giving either side an advantage, it handles augmentation carefully. The supervised model augments during training. The unsupervised side uses one augmented view for train embeddings and none for test. After embeddings, the system is standardized, run PCA to nine components, and L2-normalize. $k=2$ is set for K-means after checking elbow and silhouette so it matches the “fire vs no-fire” task. It maps clus-

ters to labels by simple majority from the file names, then score each test image as `distance_to_nofire` minus `distance_to_fire`; higher means more fire-like.

Decisions follow the same rule on both sides: it tunes a threshold on held-out data, freeze it, and then apply it to the test set. It uses F-beta because catching fires matters more than perfect precision: `beat=2.5` for the supervised model and `beta=1.5` for the unsupervised one. After that, it reevaluate, show eight sample decisions, and run a tiny “drone” simulation that prints DROP/EXIT. It is not a real deployment it is just a simple, honest way to see how both methods behave on exactly the same tasks.

(a) Supervised pipeline



(b) Unsupervised pipeline

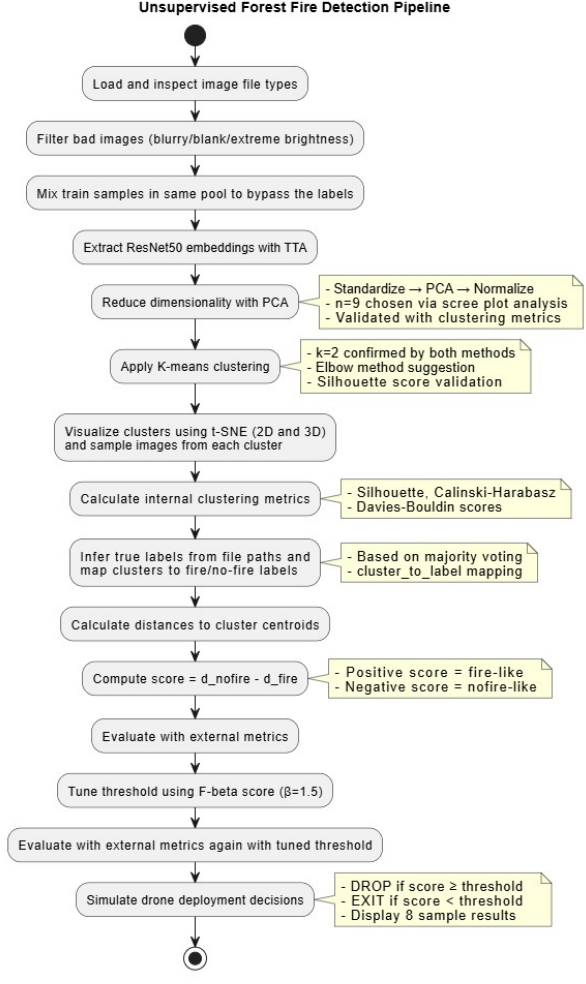


Figure 3.6: Side-by-side comparison of supervised and unsupervised wildfire pipelines.

3.4 Preprocessing

In this step of preprocessing, the system makes the comparison fair, clean, and easy to repeat. It uses the same data sources for both pipelines, only uses the test set for the final evaluation, and makes sure that the images are all the same so that they can be compared. It also delete images that cannot be used to reduce noise. The goal is

to eliminate data problems so that any differences in results are due to the methods themselves and not to inconsistent inputs.

3.4.1 Shared

Identical train and test sources:

The supervised and unsupervised notebooks both use the same data sources, and the test split is only used for final evaluation.

Listing 3.1: Dataset paths for training and testing

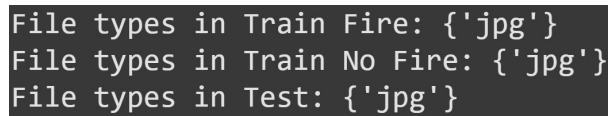
```
1 train_fire_path = '/content/Forest Fire Dataset/Training/fire'  
2 train_nofire_path = '/content/Forest Fire Dataset/Training/nofire'  
3 test_path = '/content/Forest Fire Dataset/Testing'
```

File-type sanity check

A code snippet was executed to verify file types, confirming that all images are JPG across Train/Fire, Train/No-Fire, and Test.

Listing 3.2: Sanity check: list unique file types per split

```
1 linenos,breaklines,fontsize=\small]{python}
2 def list_file_types_in_directory(directory):
3     file_types = set([
4         file.split('.')[-1]
5         for file in os.listdir(directory)
6         if os.path.isfile(os.path.join(directory, file))
7     ])
8     return file_types
9
10 train_fire_file_types = list_file_types_in_directory(train_fire_path)
11 train_nofire_file_types = list_file_types_in_directory(train_nofire_path)
12 test_file_types = list_file_types_in_directory(test_path)
13
14 print("File types in Train Fire:", train_fire_file_types)
15 print("File types in Train No Fire:", train_nofire_file_types)
16 print("File types in Test:", test_file_types)
```



```
File types in Train Fire: {'jpg'}
File types in Train No Fire: {'jpg'}
File types in Test: {'jpg'}
```

Figure 3.7: File-type check confirming that Train Fire, Train No-Fire, and Test each contain only JPG images.

Pre Filtering

The same image-quality filter is applied to both pipelines prior to preprocessing. Images are not accepted if they are unreadable, corrupt, blank, too dark or bright or too blurry. This lightweight filter gets rid of images that can't be used, reduces noise and makes training and clustering more stable and fair.

Listing 3.3: Filtering corrupted images

```

1 def is_bad_image(img_rgb, file=None, min_unique=50, mean_lo=5, mean_hi=250,
  ↪ blur_thr=20.0):
2
3     if img_rgb is None:
4         print(f"[Skipped unreadable] {file}" if file else "[Skipped
  ↪ unreadable]")
5         return True
6
7     gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY) if img_rgb.ndim == 3
  ↪ else img_rgb
8
9     if np.unique(gray).size < min_unique:
10        print(f"[Skipped near-blank] {file}" if file else "[Skipped
  ↪ near-blank]")
11        return True
12
13    m = float(gray.mean())
14    if m < mean_lo or m > mean_hi:
15        print(f"[Skipped extreme brightness] {file} (mean={m:.1f})" if file
  ↪ else f"[Skipped extreme brightness] (mean={m:.1f})")
16        return True
17
18    lap_var = cv2.Laplacian(gray, cv2.CV_64F).var()
19    if lap_var < blur_thr:
20        print(f"[Skipped blurry] {file} (var={lap_var:.1f})" if file else
  ↪ f"[Skipped blurry] (var={lap_var:.1f})")
21        return True
22
23    return False

```

3.4.2 Supervised only

Loading images

Each image is read as JPG, converted from BGR to RGB, resized to 224×224 , checked with a bad-image filter, cast to float32 as required by the backbone, and labeled when applicable.

Listing 3.4: Supervised preprocessing — load images from a directory

```

1  def load_images_from_directory(directory, label=None, return_names=False, extensions=('.jpg')):
2
3      images = []
4      labels = []
5      filenames = []
6
7      for file in sorted(os.listdir(directory)):
8          if not file.lower().endswith(extensions):
9              continue
10
11         path = os.path.join(directory, file)
12         img = cv2.imread(path)
13
14         if img is None:
15             is_bad_image(None, file=path)
16             continue
17
18         img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
19         img = cv2.resize(img, (224, 224), interpolation=cv2.INTER_AREA)
20
21         if is_bad_image(img, file=path):
22             continue
23
24         images.append(img)
25         if label is not None:
26             labels.append(label)
27         filenames.append(file)
28
29     images = np.array(images, dtype='float32')
30     labels = np.array(labels, dtype='int32') if label is not None else np.array([], dtype='int32')
31     filenames = np.array(filenames, dtype=object)
32
33     if return_names:
34         return images, labels, filenames
35     return images, labels

```

Load class folders + filter blurry

Training data from train/fire are loaded with label = 1 and from train/nofire with label = 0; anomalous images are skipped to keep the training set clean.

Listing 3.5: Load Train/Fire (1) and Train/NoFire (0) with blur filtering

```

1  train_fire_images, train_fire_labels = load_images_from_directory(train_fire_path, 1)
2  train_nofire_images, train_nofire_labels = load_images_from_directory(train_nofire_path, 0)

```

Build training arrays

Fire and No-Fire images are concatenated to form `X_train`, and their labels are combined into `y_train`, yielding a unified training set.

Listing 3.6: Concatenate class tensors into a unified training set

```
1 X_train = np.concatenate((train_fire_images, train_nofire_images), axis=0)
2 y_train = np.concatenate((train_fire_labels, train_nofire_labels), axis=0)
```

Load test set (names kept)

All images from the test folder are loaded using the same preprocessing steps, without assigning labels; original filenames are retained to match predictions to source images for later reporting and inspection.

Listing 3.7: Load TEST images (keep filenames for reporting)

```
1 test_images, _, test_image_names = load_images_from_directory(
2     test_path, None, return_names=True
3 )
```

Train/Validation split

The training set is split into Train (80%) and Validation (20%) with a fixed seed (`random.state=42`) and `stratify=y_train` to preserve the class ratio.

Listing 3.8: Train/Validation split with stratification (no leakage)

```
1 X_train, X_val, y_train, y_val = train_test_split(
2     X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
3 )
```

3.4.3 Unsupervised only

Unlabeled Data Setup

An unlabeled training set was created by merging the fire and no-fire training folders, and this set was used to train the unsupervised pipeline. There were no labels used anywhere in the pipeline. The model only sees images and has to find structure on its own. For the final evaluation a separate labeled test set was kept separate. So there are a lot of unlabeled images for fitting and a small labeled split that is only used to choose a threshold and report final metrics.

Listing 3.9: Listing and validating image file paths for training and testing datasets

```
1 def list_images(root, exts=(".jpg",)):  
2     return [p for p in glob.glob(os.path.join(root, "*"))  
3             if os.path.isfile(p) and p.endswith(exts)]  
4  
5 train_paths = list_images(train_fire_path) + list_images(train_nofire_path)  
6 test_paths  = list_images(test_path)  
7  
8 print(f"Train: {len(train_paths)} | Test: {len(test_paths)}")
```

Image Loading and Normalization

OpenCV loads each image and it raises a `ValueError` if the file is missing or broken. Then, the image is resized to fit backbone. To keep the channel semantics correct, The color space was changed from bgr to rgb because opencv reads in bgr order and backbone expects rgb. A quality gate(process input function) skips the inputs that can't be used, like frames that are empty, all black or white, the wrong shape, or Finally, data are cast to float32, consistent with the supervised pipeline.

Listing 3.10: Image Loader with Resizing, BGR to RGB Conversion, and Quality Checks

```
1 def load_rgb_cv2(p, size):
2
3     img = cv2.imread(p)
4     if img is None:
5         raise ValueError(f"Failed to read image: {p}")
6     img = cv2.resize(img, size, interpolation=cv2.INTER_AREA)
7     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
8
9     if is_bad_image(img, file=p):
10         raise ValueError("bad_image")
11
12     return img.astype(np.float32)
```

3.5 Models

Two models with the same setup are compared. Both use a frozen ResNet-50 backbone with a 224×224 RGB input, this ensures a fair comparison. In the supervised path, a small classifier head is added and trained on labels. In the unsupervised path, the same backbone is used to extract embeddings. With the backbone frozen, any performance differences arise from the downstream step rather than the backbone itself.

3.5.1 Shared:

Shared Backbone and Input Size

Both notebooks use the same frozen ResNet50 as their backbone and the only differences come from the downstream step because supervised adds a small head and unsupervised uses PCA and K means.

Listing 3.11: Backbone definition

```

1 backbone = ResNet50(weights="imagenet", include_top=False,
  ↳ input_shape=(224,224,3), pooling="avg")
2 backbone.trainable = False

```

3.5.2 Supervised only

Generator and Data Flows

This code sets up the training images. First, it uses ResNet’s preprocess_input so the pictures match what the backbone expects. Then it adds small random changes (flip, rotate, zoom, brightness change) so the model keeps seeing different versions of the same image and doesn’t just memorize. Validation only keeps the clean preprocessing (no random changes) so the check is fair. Finally, it makes two data flows: a shuffled training flow (batch 32, fixed seed) and a steady validation flow (batch 32, no shuffle).

Listing 3.12: Supervised preprocessing: validation datagen and data flows

```

1 train_datagen = ImageDataGenerator(
2     preprocessing_function=preprocess_input,
3     horizontal_flip=True,
4     rotation_range=10,
5     width_shift_range=0.1,
6     height_shift_range=0.1,
7     zoom_range=0.2,
8     brightness_range=(0.9, 1.1),
9     fill_mode="nearest"
10 )
11 val_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)
12
13 train_flow = train_datagen.flow(X_train, y_train, batch_size=32, shuffle=True,
  ↳ seed=42)
14 val_flow   = val_datagen.flow(X_val,   y_val,   batch_size=32, shuffle=False)

```

Augmentation on Training visualization:



Figure 3.8: Training augmentation visualization: top row shows originals and bottom row shows randomly augmented versions.

Classification Head

A small classifier is added on top of the ResNet features to make it supervised. This top layer changes the 2048-number feature into a score between 0 and 1 (no fire and fire). This part was trained with the labels while the ResNet backbone stays still.

Listing 3.13: Custom classification head on top of the ResNet50 backbone

```
1 x = backbone.output
2 x = Dense(64, use_bias=False,
3         kernel_initializer="he_normal",
4         kernel_regularizer=tf.keras.regularizers.l2(1e-3))(x)
5 x = BatchNormalization()(x)
6 x = Activation("relu")(x)
7 x = Dropout(0.5)(x)
8 out = Dense(1, activation="sigmoid")(x)
```

Compile Setup

The feature extractor and a small classifier were combined into a single model and trained on labels with a low learning rate. Accuracy was monitored, and a binary classification loss was used. The backbone remained frozen; only the top layers were updated.

Listing 3.14: Model compilation with Adam optimizer and binary crossentropy loss

```
1 model = Model(backbone.input, out)
2 model.compile(
3     optimizer=Adam(learning_rate=2e-5),
4     loss="binary_crossentropy",
5     metrics=["accuracy"])
```

3.5.3 Unsupervised only

Embedding Extraction (original + augmented views)

Each image is loaded, resized to 224×224 in RGB, and normalized according to ResNet-50. When an augmentation count is specified, multiple augmented views of the same image (e.g., flips, rotations, shifts) are generated. All views are passed through ResNet-50, and a stable 2048-dimensional embedding is obtained by averaging their feature vectors. Corrupted files are skipped, and processed image indices are recorded. The result is a clean matrix containing one embedding per image.

Listing 3.15: Extracting ResNet embeddings with tta mean using opencv loader

```

1  def extract_ResNet_embeddings_mean_cv2(paths, n_aug=1, seed=None,
2                                     aug_datagen=None, eval_datagen=None):
3
4      rng = np.random.RandomState(seed) if seed is not None else None
5      embs, kept_idx = [], []
6
7      for i, p in enumerate(tqdm(paths, desc="Embeddings (TTA-mean)")):
8          try:
9              arr0 = load_rgb_cv2(p, (224, 224))
10
11              views = []
12              x0 = eval_datagen.standardize(arr0.copy())
13              views.append(x0)
14
15              for _ in range(int(n_aug)):
16                  s = None if rng is None else int(rng.randint(0, 2**31 - 1))
17                  aug = aug_datagen.random_transform(arr0.copy(), seed=s)
18                  aug = aug_datagen.standardize(aug)
19                  views.append(aug)
20
21              batch = np.stack(views, axis=0).astype(np.float32)
22              vecs = backbone.predict(batch, verbose=0)
23              embs.append(vecs.mean(axis=0))
24              kept_idx.append(i)
25
26          except Exception as e:
27              tqdm.write(f"Skip {p}: {e}")
28
29      if not embs:
30          raise ValueError("No embeddings produced.")
31
32      return np.vstack(embs), np.array(kept_idx, dtype=int)

```

Two Stream Preprocessing: Train Augment, Test Clean

This code prepares images for making embeddings. It applies the same ResNet preprocess_input. For training images, it adds small random changes to make the features steady even if the picture shifts a little. For evaluation/test, it keeps only the clean preprocessing (no random changes) so results are consistent and fair.

Listing 3.16: Training and evaluation data generators

```

1  train_datagen = ImageDataGenerator(
2      preprocessing_function=preprocess_input,
3      horizontal_flip=True,
4      rotation_range=10,
5      width_shift_range=0.1,
6      height_shift_range=0.1,
7      zoom_range=0.2,
8      brightness_range=(0.9, 1.1),
9      fill_mode="nearest")
10
11 eval_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

```

Features are extracted for the training set using one additional augmented copy per image, with a fixed random seed to ensure reproducibility. For the test set, no augmentation is applied—only the original images are used to reflect real-world operation. Finally, counts are printed to verify that the number of feature vectors matches the expected number of images in each split.

Listing 3.17: Embedding Extraction (TTA for train, deterministic for test)

```

1  train_embeds, train_valid_idx = extract_ResNet_embeddings_mean_cv2(
2      train_paths, n_aug=1, seed=42, aug_datagen=train_datagen,
3      ↪ eval_datagen=eval_datagen)
4
5  test_embeds, test_valid_idx = extract_ResNet_embeddings_mean_cv2(
6      test_paths, n_aug=0, seed=None, aug_datagen=eval_datagen,
7      ↪ eval_datagen=eval_datagen)
8
9  print("\nTrain embeds:", train_embeds.shape, "| Test embeds:",
10 ↪ test_embeds.shape)

```

```

Embeddings (TTA-mean): 5%|██████████| 75/1520 [00:18<02:09, 11.13it/s][Skipped blurry] /content/Forest Fire Dataset/Training/fire/fire_0047.jpg (var=16.8)
Skip /content/Forest Fire Dataset/Training/fire/fire_0047.jpg: bad_image
Embeddings (TTA-mean): 27%|██████████| 413/1520 [00:56<01:36, 11.48it/s][Skipped blurry] /content/Forest Fire Dataset/Training/fire/fire_0168.jpg (var=11.2)
Skip /content/Forest Fire Dataset/Training/fire/fire_0168.jpg: bad_image
Embeddings (TTA-mean): 36%|██████████| 543/1520 [01:11<01:25, 11.37it/s][Skipped blurry] /content/Forest Fire Dataset/Training/fire/fire_0934.jpg (var=10.8)
Skip /content/Forest Fire Dataset/Training/fire/fire_0934.jpg: bad_image
Embeddings (TTA-mean): 57%|██████████| 863/1520 [01:48<00:57, 11.43it/s][Skipped blurry] /content/Forest Fire Dataset/Training/nofire/nofire_0419.jpg (var=7.8)
Skip /content/Forest Fire Dataset/Training/nofire/nofire_0419.jpg: bad_image
Embeddings (TTA-mean): 59%|██████████| 897/1520 [01:51<00:57, 10.80it/s][Skipped blurry] /content/Forest Fire Dataset/Training/nofire/nofire_0421.jpg (var=15.8)
Skip /content/Forest Fire Dataset/Training/nofire/nofire_0421.jpg: bad_image
Embeddings (TTA-mean): 100%|██████████| 1520/1520 [03:03<00:00, 8.28it/s]
Embeddings (TTA-mean): 3%|██████████| 13/380 [00:06<00:59, 6.15it/s][Skipped blurry] /content/Forest Fire Dataset/Testing/nofire_0574.jpg (var=18.3)
Skip /content/Forest Fire Dataset/Testing/nofire_0574.jpg: bad_image
Embeddings (TTA-mean): 100%|██████████| 380/380 [00:41<00:00, 9.26it/s]
Train embs: (1515, 2048) | Test embs: (379, 2048)

```

Figure 3.9: Embedding extraction log

3.6 Training

Training was carried out with the same ResNet50 both for the supervised and unsupervised approaches. For the first one, there is a small classifier to the frozen backbone, it only uses light augmentation on the training set, keeps the validation set clean, and stops early when the validation loss stops getting better. For the unsupervised approach, it uses training stats to standardize, learn PCA on training and keep 9 components, L2-normalize, and then uses elbow and silhouette on training to find the number of clusters. Both point to 2. It fits K-Means to the training data and only give clusters to the test data to avoid leakage. I don't add anything to the validation or test data.

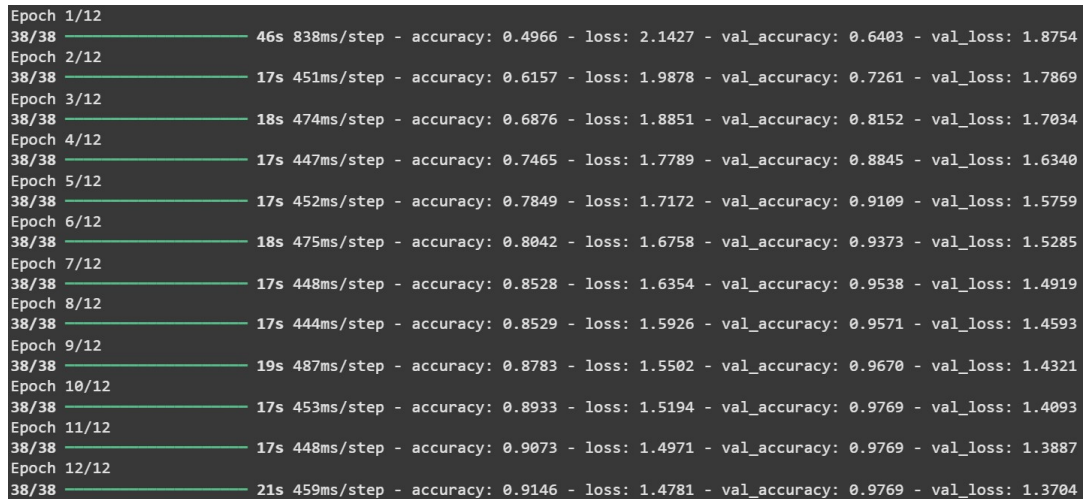
3.6.1 Supervised only

Training and Validation

Training was run for 12 epochs and evaluated on the validation set. Early stopping was employed to prevent overfitting: when validation performance ceased improving, training halted and the weights from the best epoch were restored, yielding a model that generalizes well while remaining efficient.

Listing 3.18: Model training with early stopping callback

```
1 callbacks = [tf.keras.callbacks.EarlyStopping(monitor="val_loss", mode="min",
↪ patience=2, restore_best_weights=True)]
2
3 history = model.fit(
4     train_flow,
5     validation_data=val_flow,
6     epochs=12,
7     callbacks=callbacks,
8     verbose=1)
```



Epoch	38/38	Time	Steps/Sec	Accuracy	Loss	Val Accuracy	Val Loss
1/12	38/38	46s	838ms/step	0.4966	2.1427	0.6403	1.8754
2/12	38/38	17s	451ms/step	0.6157	1.9878	0.7261	1.7869
3/12	38/38	18s	474ms/step	0.6876	1.8851	0.8152	1.7034
4/12	38/38	17s	447ms/step	0.7465	1.7789	0.8845	1.6340
5/12	38/38	17s	452ms/step	0.7849	1.7172	0.9109	1.5759
6/12	38/38	18s	475ms/step	0.8042	1.6758	0.9373	1.5285
7/12	38/38	17s	448ms/step	0.8528	1.6354	0.9538	1.4919
8/12	38/38	17s	444ms/step	0.8529	1.5926	0.9571	1.4593
9/12	38/38	19s	487ms/step	0.8783	1.5502	0.9670	1.4321
10/12	38/38	17s	453ms/step	0.8933	1.5194	0.9769	1.4093
11/12	38/38	17s	448ms/step	0.9073	1.4971	0.9769	1.3887
12/12	38/38	21s	459ms/step	0.9146	1.4781	0.9769	1.3704

Figure 3.10: Training progress across 12 epochs

Training and Validation Curves

The model did a great job. By epoch 12, the accuracy had steadily gone up and reached a high level on the validation set. Loss kept going down on both the train and the validation sets and the curves stayed close together with validation being a little higher than train (because of augmentation and regularization). This shows that the model was able to generalize well and didn't overfit.

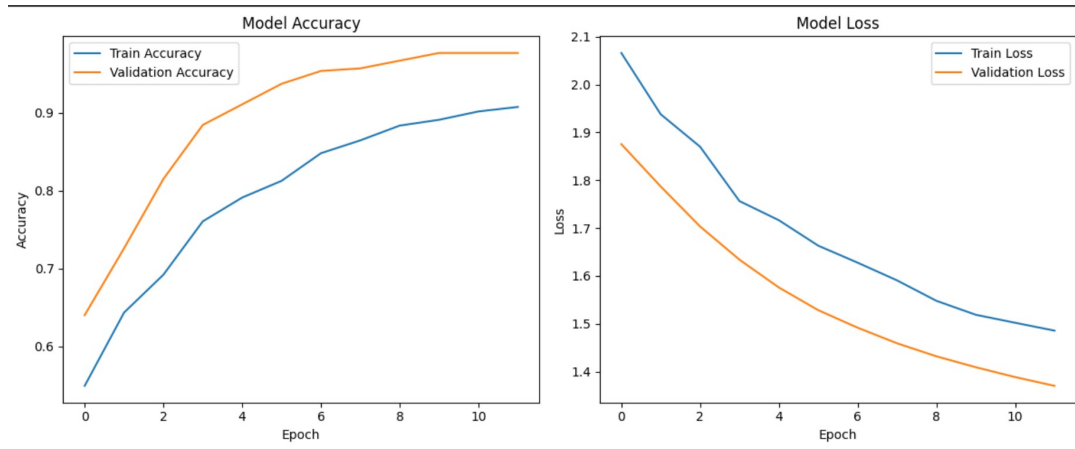


Figure 3.11: Training/validation curves: accuracy rises and loss falls, with close train–val traces indicating good generalization and minimal overfitting.

3.6.2 Unsupervised only

Feature Scaling

Embeddings are standardized by fitting a scaler on the training features and applying it to both train and test. This places each feature on a common scale (zero mean, unit variance), improving PCA and K-Means performance and preventing information leakage from the test set.

Listing 3.19: Standardization of Training and Test Embeddings

```

1 scaler = StandardScaler().fit(train_embeds)
2 train_z = scaler.transform(train_embeds)
3 test_z = scaler.transform(test_embeds)

```

Embedding Compression via PCA

The 2048-D ResNet features are reduced with PCA fitted on the training set only. Based on the scree plot, nine components are selected at the elbow, where additional components contribute little information. Both train and test features are projected into this

9-D space and L2-normalized so that distances behave like cosine similarity—emphasizing direction over magnitude—which improves clustering.

Listing 3.20: Fitting PCA on Standardized Training Embeddings

```
1  pca_full = PCA(svd_solver="full", random_state=42).fit(train_z)
```

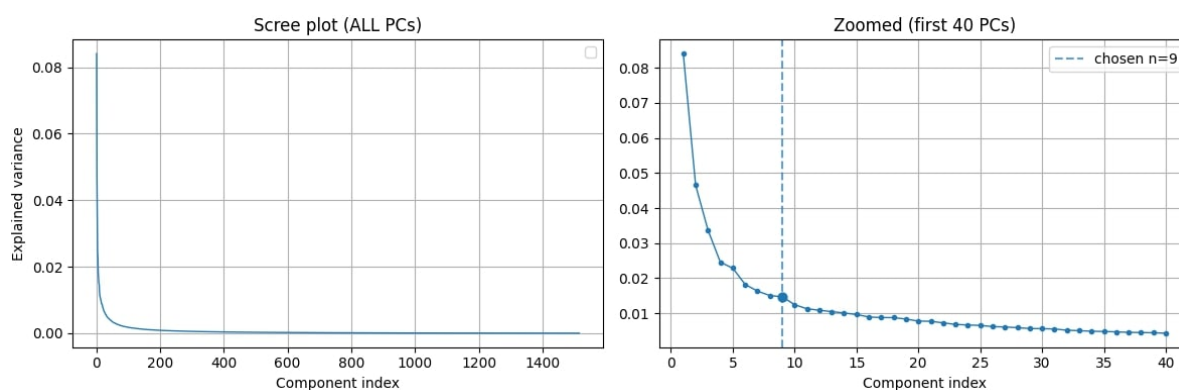


Figure 3.12: Scree plot of PCA explained variance showing an elbow around 9 components, which are selected for projection.

Listing 3.21: Applying PCA Transformation and Normalization on Training and Test Sets

```
1  X_tr_input = train_z
2  VALID_IDX = np.arange(X_tr_input.shape[0])
3
4  pca = PCA(n_components=9, svd_solver="full", random_state=42)
5  pca.fit(X_tr_input)
6
7  X_train_pca = normalize(pca.transform(X_tr_input))
8  X_test_pca = normalize(pca.transform(test_z))
9
10 train_valid_idx = VALID_IDX
11
12 print(f"Fitting PCA on RAW TRAIN, samples={X_tr_input.shape[0]}")
13 print("TRAIN:", X_train_pca.shape, "| TEST:", X_test_pca.shape)
```

```
Fitting PCA on RAW TRAIN, samples=1515
TRAIN: (1515, 9) | TEST: (379, 9)
```

Figure 3.13: Console output confirming PCA fitted on 1,515 training samples and projection to 9 dimensions for both Train and Test.

K-Means Cluster Count

t-SNE is used to project the training embeddings into 2D and 3D, with points colored by their assigned cluster to assess how well K-means captures the geometry. Perplexity is chosen relative to dataset size so each point has a reasonable neighborhood, avoiding views that are either too fragmented or overly smooth. Because the PCA features are L2-normalized, cosine distance is used. For stability, PCA initialization is applied, the learning rate is set to auto, and optimization runs for 2000 iterations. The resulting 2D and 3D plots show two compact, well-separated clouds with balanced counts, visually supporting the earlier choice of $k = 2$. This figure is qualitative—cluster separation is inspected visually—whereas the choice of k and quantitative assessment come from the elbow and silhouette analyses, not from t-SNE.

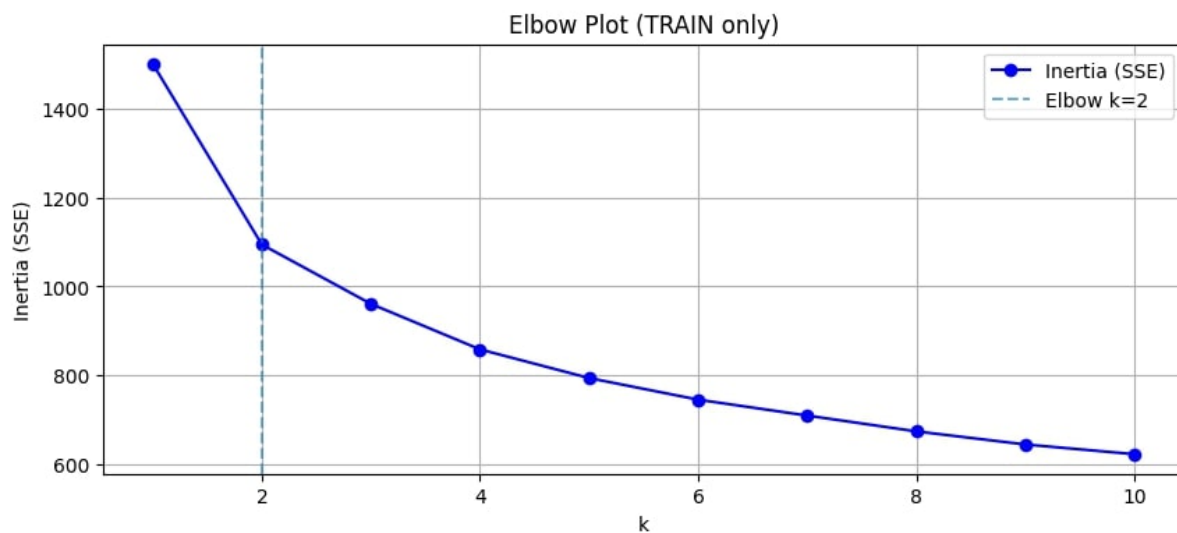


Figure 3.14: Elbow curve on training embeddings: the largest SSE drop is from $k=1$ to $k=2$ and the curve flattens afterward, indicating $k=2$.

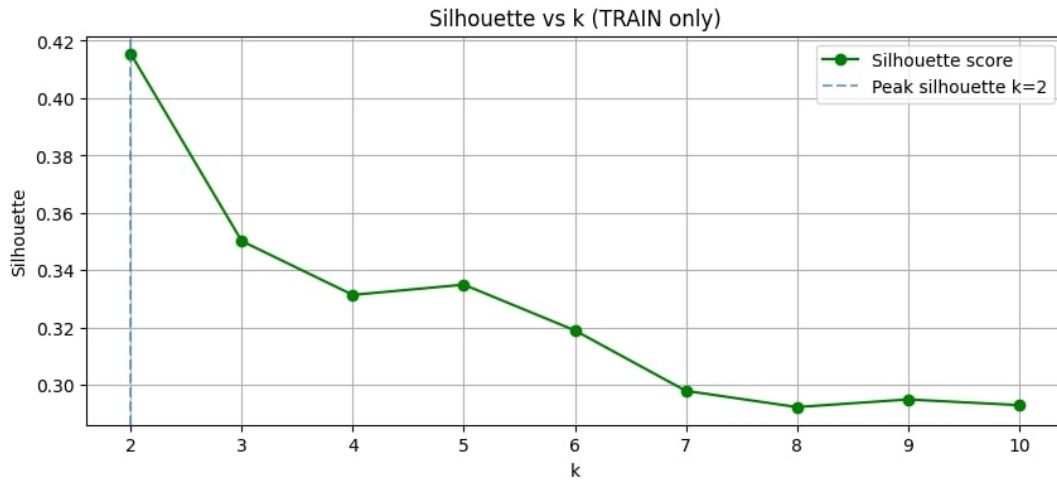


Figure 3.15: Silhouette scores on training embeddings peak at $k=2$.

Listing 3.22: K-Means clustering on PCA-transformed Embeddings

```

1 k = 2
2 kmeans = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_train_pca)
3 train_clusters = kmeans.labels_
4 test_clusters = kmeans.predict(X_test_pca)

```

t-SNE Visualisation:

I use t-SNE to project the training embeddings to 2D and 3D and color the points by their cluster to see how well K Means learned the geometry. I choose perplexity based on the size of the set so that each point has a reasonable number of neighbors to look at. This stops the view from being too broken up or too smooth. Since the PCA features were L2 normalized, I use cosine distance. To make things stable, I start with a PCA initialization, set the learning rate to auto, run for 2000 iterations. The 2D and 3D plots show two small, well-separated clouds with balanced counts. This visually backs up the choice of $K=2$ made earlier. This figure is qualitative; it helps make sure that the clusters are separate. However, the choice of K and the metrics come from the elbow and silhouette analyses, not from t SNE.

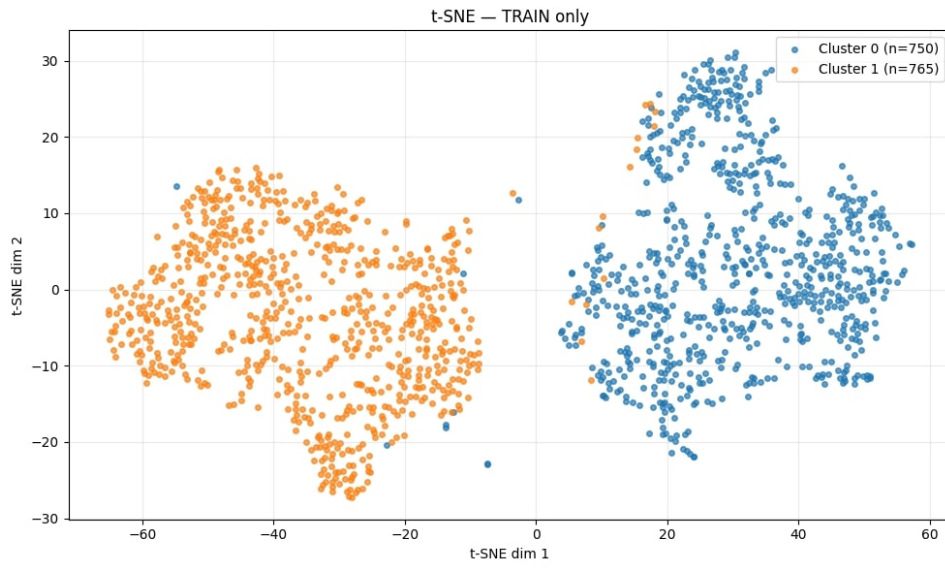


Figure 3.16: t-SNE (2D) of training embeddings.

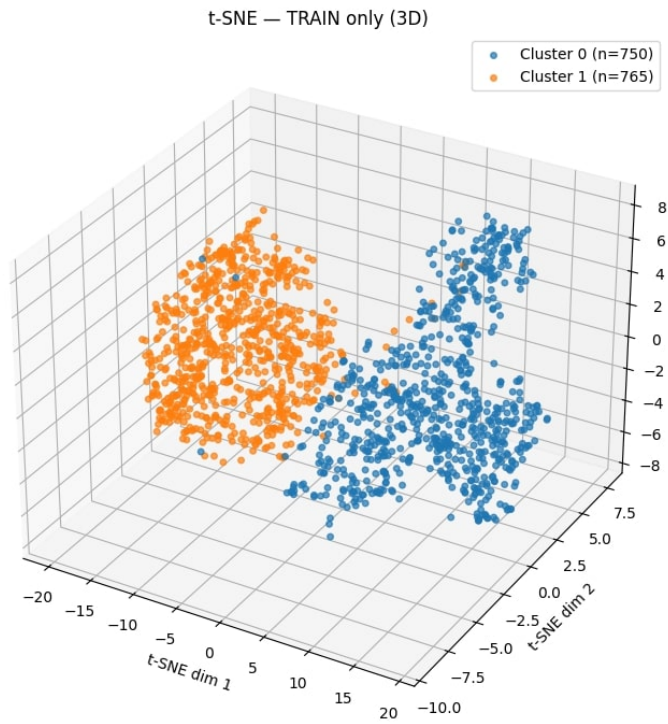


Figure 3.17: t-SNE (3D) of training embeddings.

Train-only Samples per Cluster

After PCA + K-Means ($k=2$), random training images from each cluster were shown. It's clear that Cluster 0 is fire or smoke and Cluster 1 is not fire (forests or mountains). The grouping looks good, with clear separation and no obvious mistakes. This makes us sure that the unsupervised pipeline is working as it should. This is only for training; the test set stays the same for the final metrics.

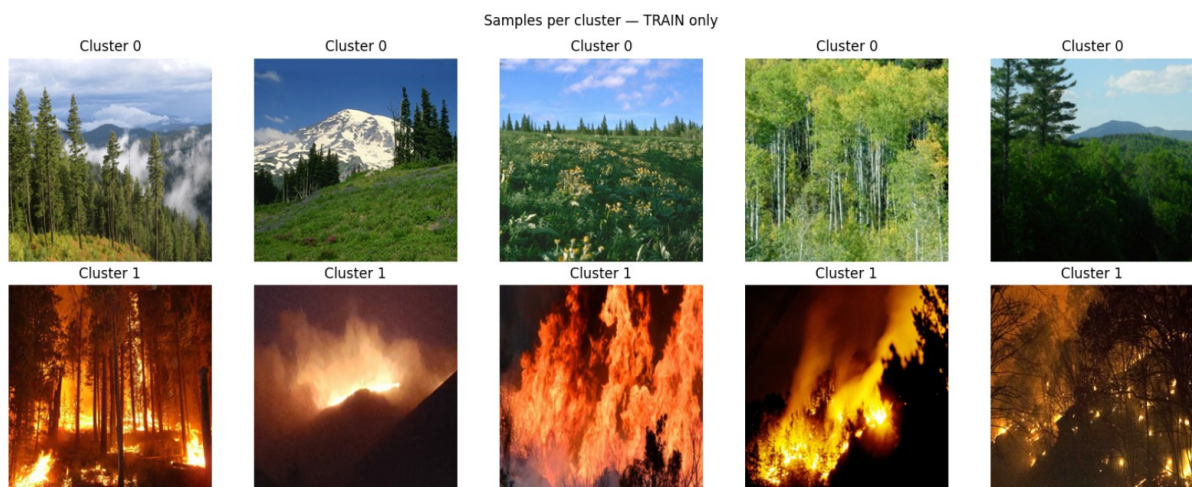


Figure 3.18: Random training examples per K-Means cluster ($k=2$), showing clear visual separation between non fire and fire scene.

3.7 Evaluation:

To be fair, both pipelines are tested on the same set of data that was not used for training. The usual metrics are reported at 0.5 and at a threshold chosen before touching test on the supervised side. First, the quality of the clusters is checked on the unsupervised side. Then, train only is used to map the clusters to labels, a simple cutoff is set, and the same metrics are reported. The goal is a clean, leak-free comparison that makes it easier to catch fires.

3.7.1 Supervised only:

Test Set Cleaning and Labeling

Only files that are clearly named (fire vs. nofire) are kept; anything unclear is discarded, and ground-truth labels are taken from the filenames. This prevents mislabeled or unknown files from affecting the evaluation and makes the reported test metrics reliable.

Listing 3.23: Filtering and labeling test dataset images

```
1 keep_mask = np.array([n.lower().startswith(("fire_", "nofire_")) for n in
   ↪ test_image_names])
2
3 names_eval = np.array(test_image_names)[keep_mask]
4 test_images_eval = test_images[keep_mask]
5
6 y_test_eval = np.array([1 if n.lower().startswith("fire_") else 0 for n in
   ↪ names_eval], dtype=int)
7
8 print(f"Test samples used for eval: {len(y_test_eval)}
   ↪ (fire={y_test_eval.sum()}, nofire={(y_test_eval==0).sum()}))")
```

Test Evaluation:

The model was first tested on the test set with the default threshold of 0.5. The classification report that shows that both classes did good. The no fire class had a precision of 0.98 and a recall of 0.93 and fire class had a precision of 0.93 and a recall of 0.98. These results show that the model can reliably tell the difference between cases where there is a fire and cases where there isn't one, even before any threshold optimization.

Listing 3.24: Classification report and evaluation metrics

```
1 test_probs = model.predict(preprocess_input(test_images_eval,
    ↪ verbose=0).ravel())
2 test_pred = (test_probs >= 0.5).astype(int)
3 print("\nClassification Report:")
4 print(classification_report(y_test_eval, test_pred, target_names=["No Fire",
    ↪ "Fire"], digits=4))
```

Classification Report:				
	precision	recall	f1-score	support
No Fire	0.9832	0.9312	0.9565	189
Fire	0.9350	0.9842	0.9590	190
accuracy			0.9578	379
macro avg	0.9591	0.9577	0.9577	379
weighted avg	0.9591	0.9578	0.9578	379

Figure 3.19: Supervised classification report at threshold 0.5

The corresponding confusion matrix gives more information about how the right and wrong predictions are spread out. The model correctly identified 176 No Fire and 187 Fire cases out of 379 test samples, but it incorrectly identified 13 No Fire and 3 Fire cases. While the performance is generally good, the number of false negatives (3 fire events incorrectly predicted as no fire) could still be lower, since not finding real fires can have worse effects than raising a false alarm. This makes it necessary to later adjust the threshold to favor recall on the fire class.

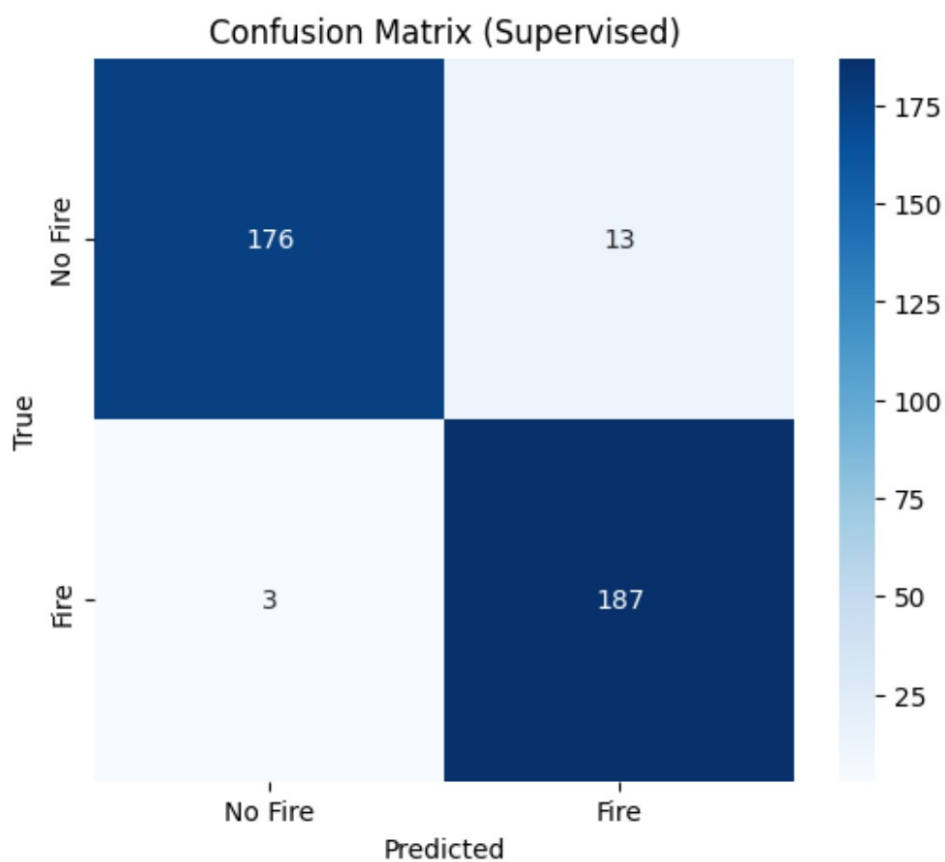


Figure 3.20: Confusion matrix on supervised model with threshold 0.5.

3.7.2 Unsupervised only:

Internal Clustering Metrics (train vs test):

Three no label metrics are checked on the PCA+L2 features, which are Silhouette (cosine), Calinski Harabasz, and Davies Bouldin (both Euclidean). Higher is better for Silhouette/CH; lower is better for DB. To compare CH fairly, the size of train is matched to test by subsampling train and averaging, then all metrics are computed on train and on test using only the predicted cluster IDs (no refit on test). Fortunately, results are very close: Silhouette 0.527 became 0.507, DB 1.33 became 1.27, CH = 215 (train) vs 208 (test). This means the clusters stay compact and well separated on unseen data, supporting $K = 2$.

Listing 3.25: Internal Clustering Evaluation Metrics on Training and Test Sets

```
1 sil_tr = silhouette_score(X_train_pca, train_clusters, metric="cosine")
2 db_tr = davies_bouldin_score(X_train_pca, train_clusters)
3
4 sil_te = silhouette_score(X_test_pca, test_clusters, metric="cosine")
5 ch_te = calinski_harabasz_score(X_test_pca, test_clusters)
6 db_te = davies_bouldin_score(X_test_pca, test_clusters)
7
8 n_te = X_test_pca.shape[0]
9 R = 20
10 rng = np.random.RandomState(0)
11 ch_tr_vals = []
12 for _ in range(R):
13     idx = rng.choice(X_train_pca.shape[0], size=n_te, replace=False)
14     ch_tr_vals.append(calinski_harabasz_score(X_train_pca[idx], train_clusters[idx]))
15 ch_tr_mean = float(np.mean(ch_tr_vals))
16 ch_tr_std = float(np.std(ch_tr_vals))
17
18 internal_metrics = pd.DataFrame({
19     "Metric": ["Silhouette", "Calinski-Harabasz", "Davies-Bouldin"],
20     "Train": [round(sil_tr,4), f"{ch_tr_mean + ch_tr_std:.6f}", round(db_tr,4)],
21     "Test": [round(sil_te,4), round(ch_te,2),round(db_te,4)]
22 })
23 internal_metrics
```

	Metric	Train	Test
0	Silhouette	0.5027	0.507200
1	Calinski-Harabasz	215.526594	208.990005
2	Davies-Bouldin	1.3309	1.279300

Figure 3.21: Internal clustering metrics.

Mapping Clusters to Classes for Evaluation:

K-Means gives cluster IDs that don't mean anything on their own. To use supervised metrics to evaluate, only the training set is used first to match each cluster to a class. The true labels of the training images for each cluster are examined and the one that gets the most votes is taken. A cluster is called fire if at least half of them are fire; if not, it is called no-fire. This gives a fixed cluster-to-class mapping without changing the test set, so there is no leakage. This mapping is then used to change the cluster IDs into predicted labels for both the training and testing sets. Accuracy, precision, recall, F1, and the rest are then reported on the test set. This is how supervised metrics are used to judge an unsupervised pipeline and make sure the evaluation is fair.

Listing 3.26: Inferring ground-truth labels from file paths and mapping clusters to classes

```
1 def infer_label_from_path(p: str):
2     if "nofire" in p: return 0
3     if "/fire/" in p or "fire" in p: return 1
4     return None
5
6 y_train_true = np.array([infer_label_from_path(train_paths[i]) for i in
7     ↪ train_valid_idx])
8
9 y_test_true = np.array([infer_label_from_path(test_paths[i]) for i in
10    ↪ test_valid_idx])
11
12 cluster_to_label = {}
13 for cid in range(kmeans.n_clusters):
14     mask = (train_clusters == cid)
15     cluster_to_label[cid] = int(y_train_true[mask].mean() >= 0.5)
16
17 print("Cluster→Class mapping:", cluster_to_label)
```



Cluster→Class mapping: {0: 0, 1: 1}

Figure 3.22: Cluster→class mapping learned on training data by majority vote.

Distance-based scoring to fire vs no fire:

Each image's PCA feature is converted into two distances: one to the fire centroid and one to the no fire centroid. For each image, a score is defined as (distance to no fire) - (distance to fire). If the score is positive, the image is closer to fire; if it's negative, it's closer to no fire. This score is computed for both train and test.

Listing 3.27: Compute signed class scores from K-Means centroid distances

```

1  d_tr = kmeans.transform(X_train_pca)
2  d_te = kmeans.transform(X_test_pca)
3
4  fire_ids = [c for c, lab in cluster_to_label.items() if lab == 1]
5  nofire_ids = [c for c, lab in cluster_to_label.items() if lab == 0]
6  assert len(fire_ids) > 0 and len(nofire_ids) > 0
7
8  d_fire_tr = np.min(d_tr[:, fire_ids], axis=1) if len(fire_ids)>1 else d_tr[:,
    ↪ fire_ids[0]]
9  d_nofire_tr = np.min(d_tr[:, nofire_ids], axis=1) if len(nofire_ids)>1 else d_tr[:,
    ↪ nofire_ids[0]]
10 d_fire_te = np.min(d_te[:, fire_ids], axis=1) if len(fire_ids)>1 else d_te[:,
    ↪ fire_ids[0]]
11 d_nofire_te = np.min(d_te[:, nofire_ids], axis=1) if len(nofire_ids)>1 else d_te[:,
    ↪ nofire_ids[0]]
12
13 score_tr = d_nofire_tr - d_fire_tr
14 score_te = d_nofire_te - d_fire_te

```

Test Evaluation:

A fair, zero threshold is used for the distance score. The score is (distance to no-fire) minus (distance to fire). So, if the image is closer to the fire centroid, fire is predicted, and if it is not, no-fire is predicted. There is no tuning on the test set. Using this fixed rule, the test split's precision, recall, F1, and accuracy are then reported.

Listing 3.28: Pre-tuning evaluation with closest-centroid threshold $t=0$ on TEST

```

1  y_test_pred = (score_te >= 0.0).astype(int)
2
3  print("Pre-tuning | Closest-centroid (t=0) | TEST")
4  print(classification_report(y_test_true, y_test_pred,
5                               target_names=["No Fire", "Fire"], digits=4))

```

Pre-tuning – Closest-centroid (t=0) – TEST				
	precision	recall	f1-score	support
No Fire	0.9630	0.9630	0.9630	189
Fire	0.9632	0.9632	0.9632	190
accuracy			0.9631	379
macro avg	0.9631	0.9631	0.9631	379
weighted avg	0.9631	0.9631	0.9631	379

Figure 3.23: Classification report for the unsupervised model at t=0:

The confusion matrix shows that the performance is still very good, with most points on the diagonal (TN = 182, TP = 183) and only a few mistakes (FP = 7, FN = 7). The precision and recall are about the same for both classes, and the overall accuracy is high. This shows that the clusters already match the labels well. Because missed fires cost more than false alarms, It is still a important to move threshold toward no fire

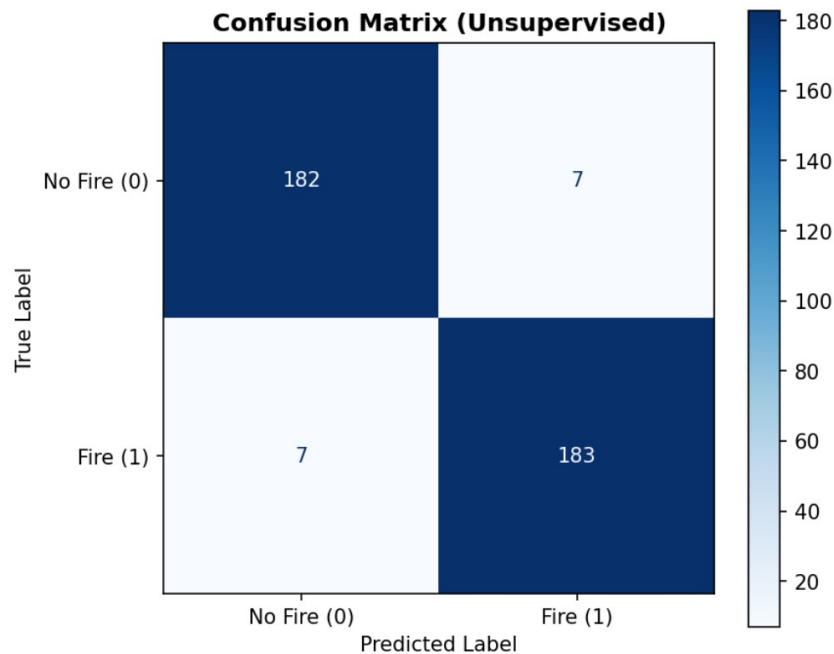


Figure 3.24: Confusion matrix for the unsupervised model at t=0:

External Clustering :

These scores compare the unsupervised clusters to the external fire and no fire labels without mapping the clusters to any class. The adjusted rand index shows that there is a lot of agreement that goes beyond chance. The V measure, which balances homogeneity and completeness, and the identical NMI both show that clusters are mostly pure and that most of the classes are grouped together, with a little mixing at the edges. Together, They show that the clustering does a good job of capturing the two class structure underneath and is in line with the thresholded classification results that were reported earlier.

	Metric	Score
0	Adjusted Rand Index (ARI)	0.8573
1	V-Measure	0.7719
2	Normalized Mutual Info (NMI)	0.7719

Figure 3.25: External clustering metrics

Demo versions I built (supervised and unsupervised)

To show how the system could guide a drone, two small demos were built. This deployment simulation is just a simple demo to show what the decision step might look like; it is not a real field simulation with flight, radios, or timing. Two quick views of the same test images are shown: in the supervised view, the model's fire probability and the fixed threshold set earlier are used to label each location as DROP or EXIT. In the unsupervised view, the risk score and the tuned cutoff (also set earlier) are used to do the same. The order is sometimes changed so that the same places are not always seen. A small gallery of up to eight images with short captions is also shown. For the unsupervised captions, the score is also changed to 0–1 so that it is easier to read (the decision does not change). A placeholder is shown instead of failing if an image is missing. The point is to show how the two pipelines would trigger DROP/EXIT decisions next to each other quickly and visually, not to simulate the whole system.

Listing 3.29: Demo function for simulating supervised version

```
1 def supervised_simulation(test_image_names, test_probs, shuffle=True):
2
3     thr = THRESH
4     probs = np.ravel(test_probs).astype(float)
5     results = [(n, float(p), ('DROP' if p >= thr else 'EXIT'))
6               for n, p in zip(test_image_names, probs)]
7     if shuffle:
8         np.random.default_rng().shuffle(results)
9     shown = results[:8]
10    for i, (name, p, decision) in enumerate(shown, start=1):
11        print(f"Location {i} | {decision.lower()} (risk={p:.4f})")
12    fig, axes = plt.subplots(2, 4, figsize=(12, 6))
13    axes = axes.ravel()
14    for ax in axes: ax.axis("off")
15
16    for idx, (name, p, decision) in enumerate(shown):
17        path = name if os.path.exists(name) else os.path.join(test_path, name) if 'test_path' in
18        ↪ globals() else name
19        img = cv2.imread(path)
20        if img is not None:
21            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
22            axes[idx].imshow(img)
23            axes[idx].set_title(f"{os.path.basename(str(name))}\n{decision.lower()} (p={p:.3f})",
24            ↪ fontsize=9)
25        else:
26            axes[idx].text(0.5, 0.5, f"Missing:\n{name}", ha="center", va="center")
27            axes[idx].set_title(f"{os.path.basename(str(name))}", fontsize=9)
28    plt.tight_layout()
29    plt.show()
30    return results, shown
```

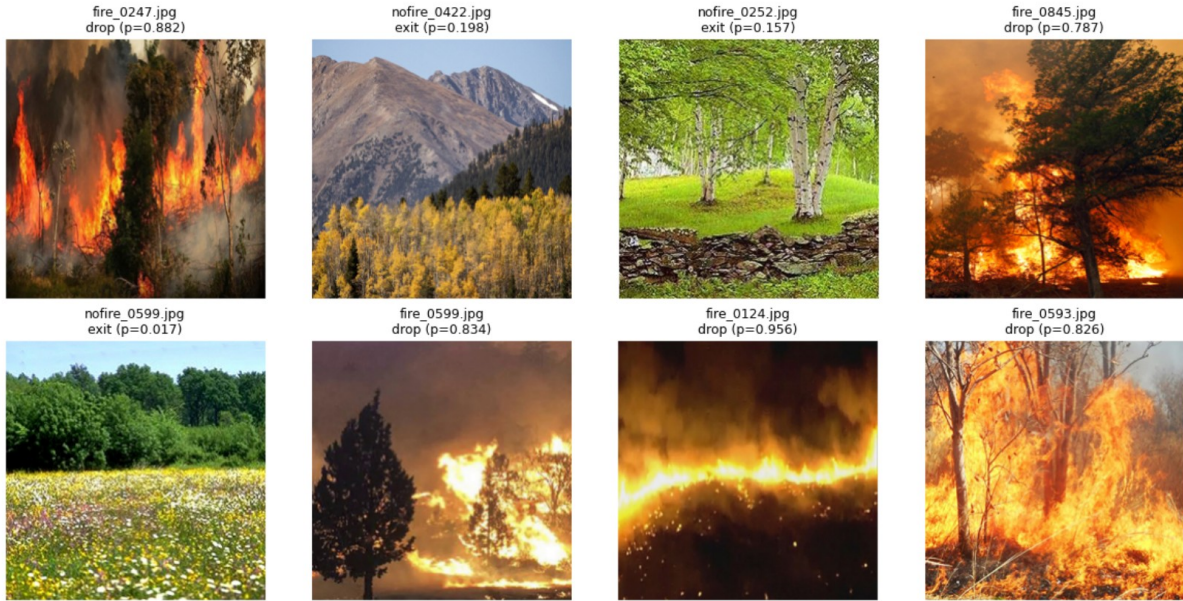


Figure 3.26: Images captured by Supervised demo

Listing 3.30: Demo function for simulating Unsupervised version

```

1  def unsupervised_simulation(test_paths, score_te, shuffle=False):
2      s = np.asarray(score_te, dtype=float).ravel()
3      thr = float(t_suggest)
4      decisions = np.where(s >= thr, "DROP", "EXIT")
5      if s.size == 0:
6          r = s
7      else:
8          lo, hi = s.min(), s.max()
9          r = np.zeros_like(s) if hi == lo else (s - lo) / (hi - lo)
10     drops = int((decisions == "DROP").sum())
11     print(f"Threshold={thr:.3f} | images={len(s)} | DROP={drops} | EXIT={len(s)-drops}")
12     for i in range(min(8, len(s))):
13         print(f"- Location {i+1}: score={s[i]:.4f}, risk={r[i]:.4f} -> {decisions[i]}")
14     idx = np.arange(len(s))
15     if shuffle:
16         np.random.shuffle(idx)
17     idx = idx[:min(8, len(idx))]
18     plt.figure(figsize=(12, 6))
19     for j, k in enumerate(idx, start=1):
20         ax = plt.subplot(2, 4, j) # literal numbers
21         ax.imshow(plt.imread(test_paths[k]))
22         ax.axis("off")
23         ax.set_title(f"Loc {k+1} | {decisions[k]} | s={s[k]:.2f} r={r[k]:.2f}", fontsize=9)
24     plt.tight_layout()
25     plt.show()
26     return list(zip([f"Location {i+1}" for i in range(len(s))],
27                    s.tolist(), r.tolist(), decisions.tolist()))
28

```



Figure 3.27: Images captured by Unsupervised demo

Chapter 4

Results and Discussion

4.1 Testbed

Experiments were run on Google Colab Pro using the default setup. Colab provided an NVIDIA T4 GPU (16 GB VRAM), about 12.7 GB RAM, and around 235 GB of temporary storage. Python 3.12 was used with TensorFlow/Keras, NumPy, scikit-learn, OpenCV, Pandas, Matplotlib, tqdm, and occasionally seaborn. CUDA and cuDNN were kept as provided by Colab (no custom installs). For reproducibility, the random seed was set to 42 in Python, NumPy, and TensorFlow. During training, GPU memory usage was typically 4–5 GB, and system RAM was 5–6 GB.

For simple data checks and preprocessing, a Windows 11 (64-bit) laptop was also used: Intel Core i7-1165G7 (4 cores / 8 threads), 16 GB RAM, and Intel Iris Xe graphics (driver 32.0.101.6987). Models were trained on the Colab GPU, with only light preprocessing performed on the laptop.

4.2 Experiments Plan

4.2.1 Threshold Tuning

Missing a fire is much worse than a false alarm. Missing data slows response, causes the fire to grow, and puts people and property at risk. A false alarm usually triggers an extra check, which is cheaper. So the system is tuned to catch almost every fire while keeping false alarms low enough to be practical.

In the validation set, many threshold values are tried and precision and recall are measured at each one. The point that gives the best F-beta score with beta greater than 1 (giving more weight to recall) is chosen. Then, that threshold is frozen and used once on the test set. The same process is used for both pipelines: for the supervised pipeline, the model's fire probability is used; for the unsupervised pipeline, its distance score is used. In both cases the cut-off is varied, the F-beta-best cut-off is chosen, it is frozen, and the test is run once. This keeps the tuning fair and focused on high recall by design.

4.2.2 Supervised

As explained in the previous paragraph, the default 0.5 threshold was not kept and the validation set was used to find a better threshold. The precision-recall curve was made from the validation scores and the cut-off that maximizes the F-beta with $\beta=2.5$, which gives more weight to recall, was chosen. This moves the decision boundary so that it catches more fires and has fewer false negatives at the cost of a few more false alarms, which is acceptable. For finding fires, it is better to raise an extra alert than to miss a fire, so the focus is placed on recall.

Listing 4.1: Validation threshold via F_β maximization ($\beta = 2.5$)

```
1 score_va = model.predict(preprocess_input(X_val), verbose=0).ravel()
2
3 BETA = 2.5
4 P_val, R_val, T_val = precision_recall_curve(y_val.astype(int), score_va)
5 Pv, Rv = P_val[:-1], R_val[:-1]
6
7 fbeta = (1 + BETA**2) * P_val * R_val / (BETA**2 * P_val + R_val + 1e-12)
8 j = int(np.argmax(fbeta))
9 THRESH = float(T_val[j])
10
11 print(f"Chosen THRESH (val, F{BETA}-max) = {THRESH}")
```

```
Chosen THRESH (val, F2.5-max) = 0.4182390868663788
```

Figure 4.1: Chosen validation threshold for the supervised model.

Precision-Recall Curve for Threshold Selection

Here, the decision threshold for the validation set is plotted: as the cut-off moves to the left, precision and recall change. The best F-beta is obtained with $\beta = 2.5$, which favors recall. The dashed line marks a cut-off to the left of 0.5 to catch more fires (fewer false negatives) while keeping precision high. The score histograms on the right are well separated. The no-fire scores are low, the fire scores are high, and the dashed line lies in the low-density gap between them. This placement shows a focus on recall with only a small increase in false alarms.

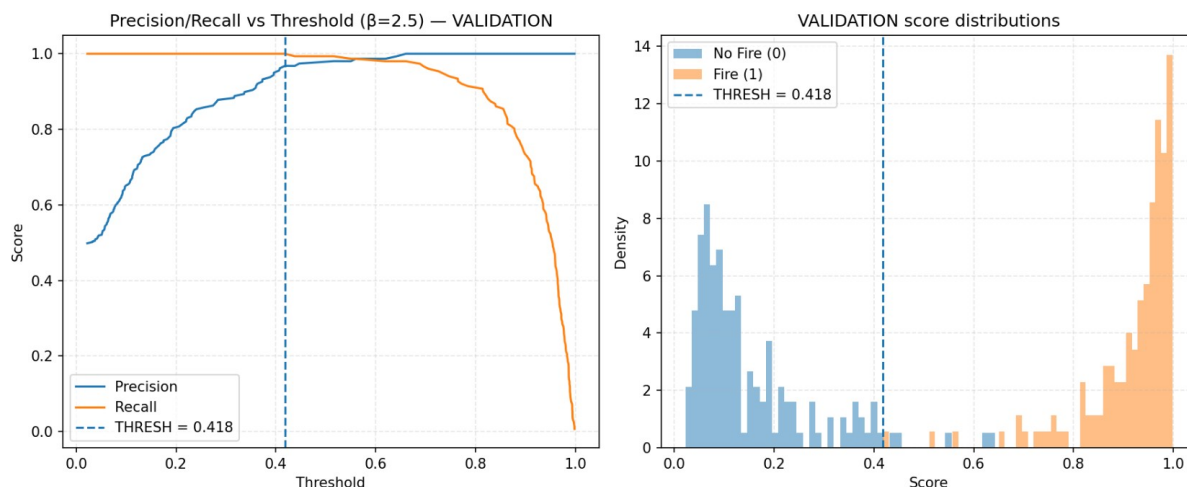


Figure 4.2: Validation precision–recall (left, $\beta = 2.5$) and score histograms (right); the dashed line marks the tuned threshold ≈ 0.418 that favors recall.

The same test probabilities are used, but instead of the default cut-off of 0.5, the tuned threshold chosen during validation is applied. Then, a new classification report is produced.

Changing the cut-off moves the decision boundary to favor recall for “Fire.” In the confusion matrix, this appears as fewer missed fires (false negatives decrease from 3 to 1) and a few more false alarms. The number of correctly detected fires increases (from 187 to 189). Overall performance remains strong and balanced, providing greater confidence in catching fires.

Listing 4.2: Test evaluation with tuned threshold suggested

```

1
2 test_pred_tuned = (test_probs >= THRESH).astype(int)
3 print("\nClassification Report:")
4 print(classification_report(y_test_eval, test_pred_tuned, target_names=["No Fire",
5   ↪ "Fire"], digits=4))

```

Classification Report:				
	precision	recall	f1-score	support
No Fire	0.9943	0.9259	0.9589	189
Fire	0.9310	0.9947	0.9618	190
accuracy			0.9604	379
macro avg	0.9627	0.9603	0.9604	379
weighted avg	0.9626	0.9604	0.9604	379

Figure 4.3: Test classification report at tuned threshold

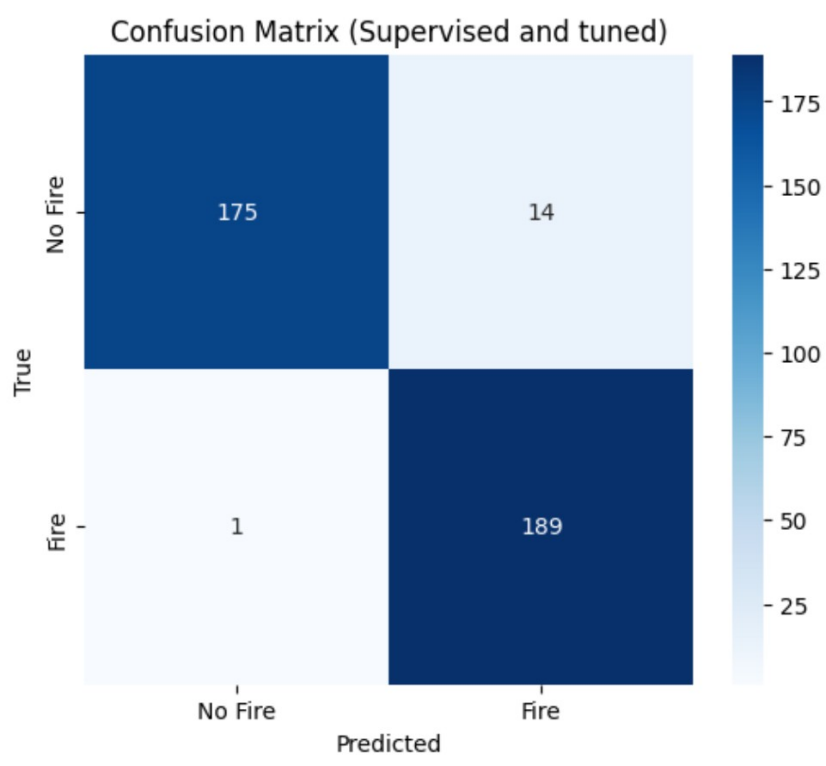


Figure 4.4: Confusion matrix (supervised, tuned)

4.2.3 Unsupervised

As noted in the previous parts, the unsupervised evaluation is made similar to the supervised model. Then, for each image, the distances to the centroids are computed. The

closest Fire centroid and the closest No Fire centroid are kept and used to make a single score that shows which side it is closer to (lower values mean Fire, higher values mean No Fire). Using this one-dimensional score, the same thresholding method as the supervised pipeline can be used. This includes choosing a threshold with an F-beta criterion to favor recall and turning scores into labels and reporting the same metrics (confusion matrix, precision, recall, F1).

Precision-Recall Curve for Threshold Selection

In this step, the continuous fire score is converted to a binary alarm. Higher scores indicate more fire-like. All candidate cut points are examined and the precision-recall curve for the fire class (label = 1) is read. Because missing a fire is more expensive than a false alarm in this case, the operating point that favors recall is chosen with $\beta=1.5$, which suggests a threshold of about -0.08. The two histograms on the right show why this cut makes sense: the orange distribution (no fire) is mostly on the left, the blue distribution (fire) is mostly on the right, and there is only a small area where they overlap. The dashed vertical line shows the chosen threshold. Placing it just to the left of zero puts it in the valley between the two clusters; it captures almost all of the fire scores, including their left-hand tail, which raises recall while still preserving most of the no-fire scores. The left plot shows this in numbers: recall stays very high around the dashed line, which is a good trade-off for precision. Overall, the histogram view shows clear class separation, and the threshold is set where the density is lowest between classes. This matches the goal of the application, which is to catch fires even if it means a few extra alarms.

Listing 4.3: Cluster distance scoring and threshold selection using F_β measure

```

1 prec, rec, thr = precision_recall_curve(y_train_true, score_tr, pos_label=1)
2 pr_auc = auc(rec, prec)
3
4 beta = 1.5
5 P, R = prec[: -1], rec[: -1]
6 f_beta = (1 + beta**2) * P * R / (beta**2 * P + R + 1e-12)
7 best_idx = int(np.argmax(f_beta))
8 t_suggest = float(thr[best_idx])

```

Suggested threshold (TRAIN, F2-max): -0.08111739158630371

Figure 4.5: Suggested threshold from training for the unsupervised

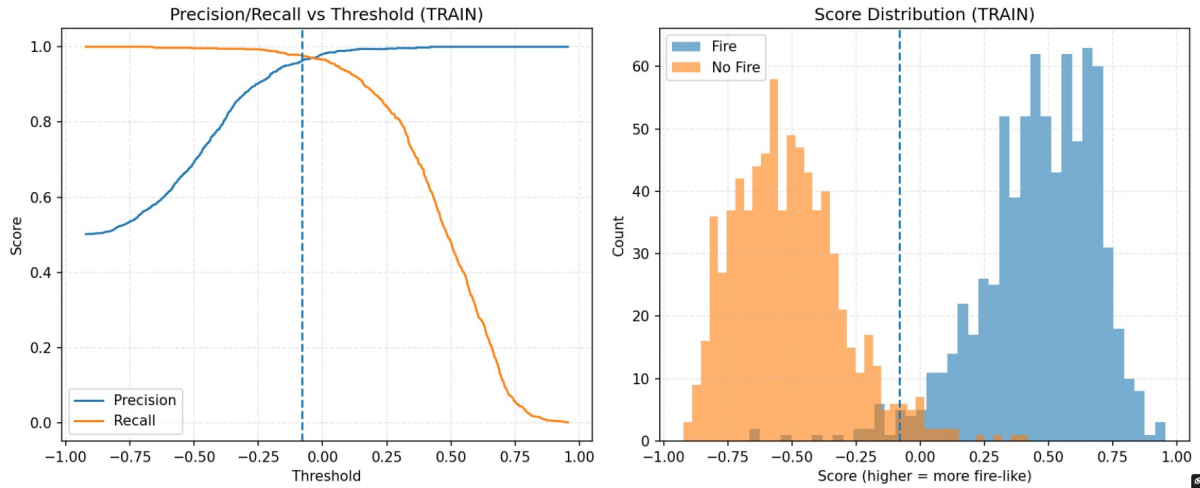


Figure 4.6: precision recall curve and score histogram for the unsupervised score; dashed line at $t \approx -0.081$ lies in the low-density gap and maximizes F_2 .

After tuning the decision threshold on training, the selected cut-off is applied to the test scores and predictions are recomputed. The classification report remains high and balanced for both classes, confirming that precision, recall, and F1 remain strong after tuning. The confusion matrix reflects the intended trade-off: it moves from the pre-tuning baseline—false negatives drop from 7 to 4 (better recall for Fire) at the cost of a small increase in false positives from 7 to 9—the right direction for a safety-critical

setting. The threshold was chosen without using test labels, so this improvement is an out-of-sample result.

Listing 4.4: Test evaluation with tuned threshold

```

1 y_test_pred_tuned = (score_te >= t_suggest).astype(int)
2 print("classification report")
3 print(classification_report(y_test_true, y_test_pred_tuned,
4                             target_names=["No Fire", "Fire"], digits=4))

```

classification report				
	precision	recall	f1-score	support
No Fire	0.9783	0.9524	0.9651	189
Fire	0.9538	0.9789	0.9662	190
accuracy			0.9657	379
macro avg	0.9661	0.9657	0.9657	379
weighted avg	0.9660	0.9657	0.9657	379

Figure 4.7: Test classification report for the tuned unsupervised model

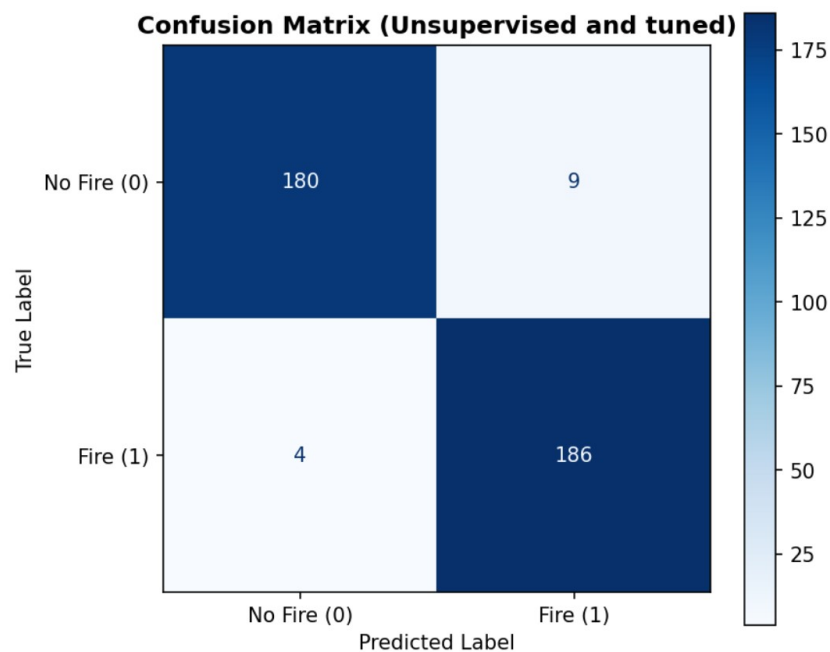


Figure 4.8: Confusion matrix (unsupervised, tuned)

4.3 Metrics

In this part, what is measured and why is described. Precision, recall, and F1 are reported for each class separately for Fire and No-Fire so the trade-offs for each class can be seen. In this setting, Fire recall is the most important because a missed fire costs more than a false alarm. The decision threshold is set on train/validation to reflect that priority, as explained in the last part, and then kept for testing. Those details are not repeated here. Macro-averaged precision, recall, and F1 are also included along with the per-class scores to give a fair, single-number summary that treats both classes the same. Accuracy is also listed, but only for the sake of completeness; in this case, it is not as important because it can hide the trade-off between recall and false alarms. Lastly, confusion matrices are shown so the numbers that make up the summary metrics can be seen.

4.3.1 Before Threshold Tuning

Both pipelines were evaluated on the test set with their default cutoffs. The supervised model uses the usual 0.5 probability and the unsupervised model uses the closest centroid rule ($t = 0$). In unsupervised model the confusion matrices is diagonal, which means the models get most images right.

Supervised (before tuning)

Strong baseline and the errors are a bit one sided 3 missed fires vs 13 false alarms. From the classification report recall is 0.9312 for No Fire and 0.9842 for Fire, and this is exactly the goal.

Unsupervised (before tuning)

Also strong and more symmetric: 7 FNs and 7 FPs. The classification report shows balanced recall for both classes (about 0.963 each), which matches the neutral cutoff and the cluster geometry.

Both models perform well at their defaults. The supervised model misses fewer fires but produces more false alarms, while the unsupervised version starts slightly more balanced at the default cutoff.

Table 4.1: Per-class metrics from classification reports — same test set (n=379)

Model	No Fire			Fire			Accuracy	Macro F1
	Prec.	Rec.	F1	Prec.	Rec.	F1		
Supervised	0.9832	0.9312	0.9565	0.9350	0.9842	0.9590	0.9578	0.9577
Unsupervised	0.9630	0.9630	0.9630	0.9632	0.9632	0.9632	0.9631	0.9631

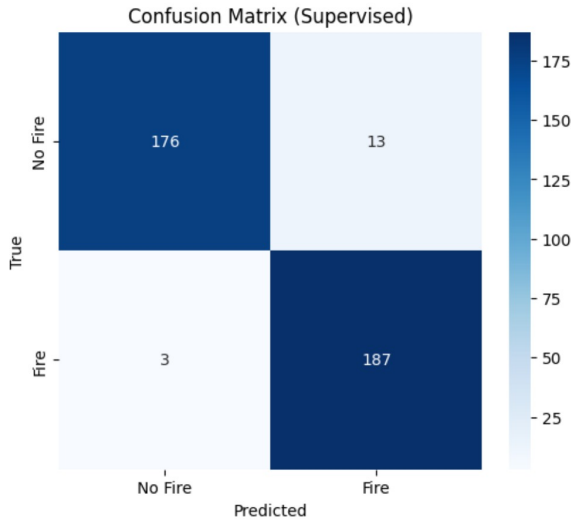


Figure 4.9: Supervised Confusion Matrix

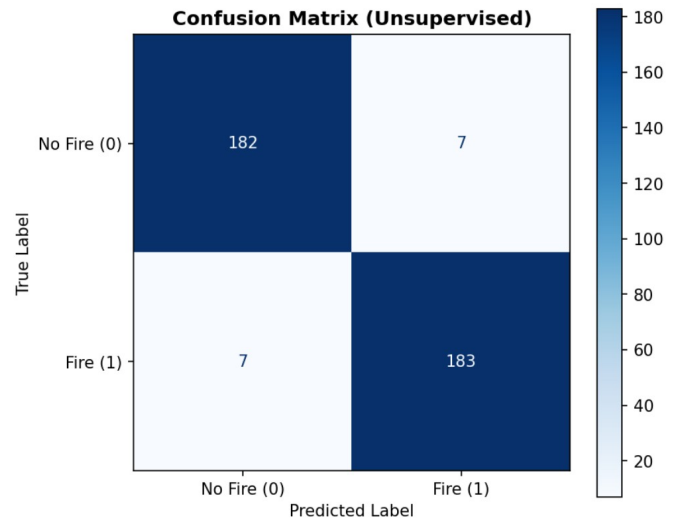


Figure 4.10: Unsupervised Confusion Matrix

4.3.2 After Threshold Tuning

A recall-aware threshold chosen on the training set is applied and kept fixed on the test set. Both models show increased recall for the Fire class.

Supervised (after tuning)

Errors shift in the intended direction, false negatives drop to 1 (this is perfect) and false positives rise to 14. From the classification report, recall is 0.9947 for Fire and 0.9259 for No Fire, so model catches more fires while keeping performance strong.

Unsupervised (after tuning)

The same pattern is observed, false negatives drop to 4 and false positives rise to 9. The report shows recall 0.9684 for Fire and 0.9365 for No Fire, which matches the goal of favoring fire recall.

So tuning the threshold improves recall for the Fire class in both pipelines with only a small cost in extra false alarms.

Table 4.2: Per-class metrics (tuned thresholds) on the same test set (n=379)

Model	No Fire			Fire			Accuracy	Macro F1
	Prec.	Rec.	F1	Prec.	Rec.	F1		
Supervised	0.9943	0.9259	0.9589	0.9310	0.9947	0.9618	0.9604	0.9604
Unsupervised	0.9672	0.9365	0.9516	0.9388	0.9684	0.9534	0.9525	0.9525

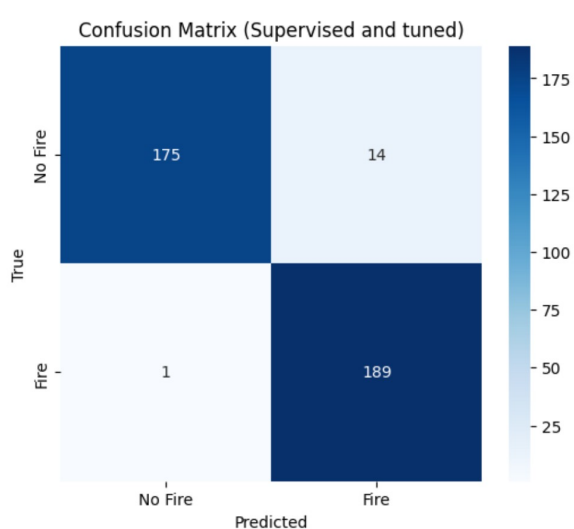


Figure 4.11: Supervised Confusion Matrix(tuned)

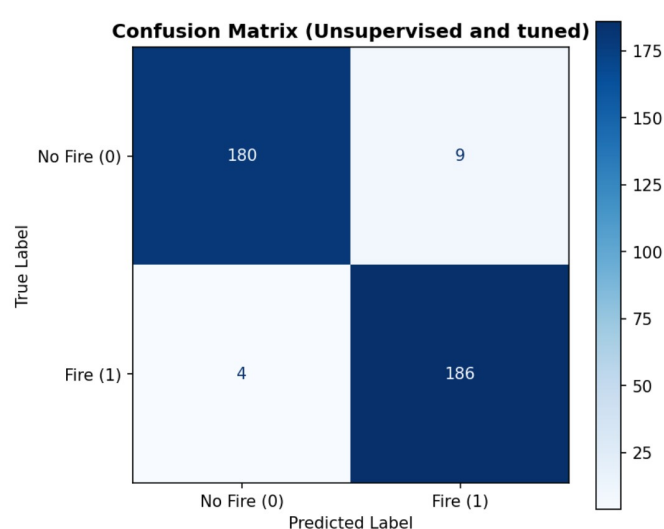


Figure 4.12: Unsupervised Confusion Matrix(tuned)

4.4 My Consideration

Even though the supervised pipeline performed better in the tests, using both on the drone is preferred to make the system more robust. The supervised model provides strong accuracy when the data resembles what it learned before, while the unsupervised

one serves as a good backup when scenes are new or labels are missing, thereby compensating for supervised model deficiencies. The supervised model can be run first and the unsupervised score used as a second check. If both indicate “fire,” it is treated as high-confidence. If they disagree, a quick verification pass (closer photo/short video) is triggered before acting. In this way, high recall is maintained while false alarms are reduced, and a fallback remains available if the main model drifts or the environment changes.

Chapter 5

Conclusion and Future Works

This work made a useful early-wildfire detection pipeline using two methods: a supervised classifier and an unsupervised embedding method. It was shown how to favor recall when missing a fire is expensive with careful preprocessing and threshold tuning. It was also shown how the system connects sensors, a base station, and a drone for targeted response. The method is possible and cheap, but it has some problems, such as not having enough different types of data, not having video or time-based tests, and not having a full field loop. These results make it clear what needs to be done next.

Next, the focus is on the following improvements:

Collect more real data

More images and short video clips from different places, seasons, times of day, and terrains will be gathered. This should reduce bias and cover edge cases like smoke without flames, sunsets, glare, fog, night scenes, and small distant fires. Labels will also be kept clean and location/time-based splits will be kept so real generalization can be tested.

Semi/self-supervised learning

When labels are limited, a semi- or self-supervised step will be added. The plan is to pretrain on unlabeled images to learn good features, then fine-tune on the labeled set. This should improve recall with fewer labels and help the model adapt faster to new environments.

Deployment on edge

The model will be converted to a light format (TFLite or ONNX), quantization and pruning will be tested, and real-time speed will be targeted on a small GPU or edge device.

Foam choice and impact

Work with experts will test different foams/retardants, measure spread pattern and coverage, and check cleanup and environmental impact. The aim is to pick an effective and safer option for early, small fires.

WSN integration

A prototype integration with a small WSN (2–3 nodes) and a base station loop can be planned to test the timing and GPS targeting from end to end, based on the architecture assumptions.

Bibliography

- [1] E. Agrillo, R. Inghilesi, A. Mercatini, A. Pezzarossa, and N. Tartaglione, “Ecosistemi terrestri ed incendi boschivi in italia: Anno 2024,” ISPRA - Istituto Superiore per la Protezione e la Ricerca Ambientale, Rome, Italy, Tech. Rep. Rapporti CSA n. 01-2025, 2025, in collaboration with SNPA; data sources include EFFIS and ECM-F4 (2020). [Online]. Available: <https://www.isprambiente.gov.it/it/attivita/Crisi-Emergenze-ambientali-e-Danno/centro-operativo-per-la-sorveglianza-ambientale/ecosistemi-ed-incendi-boschivi-in-italia>
- [2] A. A. Alkhatib, “A review on forest fire detection techniques,” *International Journal of Distributed Sensor Networks*, vol. 2014, 2014, article ID 597368. [Online]. Available: <https://journals.sagepub.com/doi/full/10.1155/2014/597368>
- [3] M. E. Alexander, “Fire behaviour as a factor in forest and rural fire suppression,” Forest and Rural Fire Scientific and Technical Series, Tech. Rep. Report No. 5; Forest Research Bulletin No. 197, 2000. [Online]. Available: https://www.scionresearch.com/_data/assets/pdf_file/0009/63945/31037-FireBehaviourSuppression.pdf
- [4] National Wildfire Coordinating Group, “Module 9: Suppression, communication, and mop up,” In *Firefighter Training Course S-130*. [Online]. Available: <https://pdf4pro.com/view/module-9-suppression-communication-and-mop-up-topic-1-69d4ba.html>
- [5] U.S. Forest Service, “Aviation annual report, calendar year 2022,” United States

- Department of Agriculture, Forest Service, Tech. Rep., 2023. [Online]. Available: <https://www.fs.usda.gov/sites/default/files/2024-02/2022-Aviation-Report.pdf>
- [6] G. Jang, J. Choi, H. Lee, W. Kang, and H. Kim, “A cost-effective unmanned aerial vehicle (uav) platform for field plant breeding applications,” *Remote Sensing*, vol. 12, no. 6, p. 998, 2020. [Online]. Available: <https://www.mdpi.com/2072-4292/12/6/998>
- [7] U.S. Forest Service, “Helicopter services hourly flight rates (2018–2021),” United States Department of Agriculture, Forest Service, Tech. Rep., 2020. [Online]. Available: https://www.fs.usda.gov/sites/default/files/2020-02/flt_chrt_awarded_2018-2021.pdf
- [8] J. Rakowska, “Best practices for selection and application of firefighting foam,” *MATEC Web of Conferences*, vol. 247, p. 00014, 2018. [Online]. Available: https://www.researchgate.net/publication/329535397_Best_practices_for_selection_and_application_of_firefighting_foam
- [9] V. Apostolopoulou-Kalkavoura, B. Qin, Z. Y. Ma, Q. Chan *et al.*, “Fire-retardant and thermally insulating phenolic–silica aerogels,” *Angewandte Chemie International Edition*, Feb. 2018. [Online]. Available: https://www.researchgate.net/publication/323346935_Fire-Retardant_and_Thermally_Insulating_Phenolic-Silica_Aerogels?enrichId=rgreq-837aedfb4e326ffb72eb1a66de39e36b-XXX&enrichSource=Y292ZXJQYWdlOzMzM0NjkzNTtBUzoxMTUwMzczMjI2MTM1NTUyQDE2NTEyODE3NTI3D&el=1_x.2
- [10] L. Li, H. Zhang, Q. Xie, L. Chen, and C. Xu, “Experimental study on fire hazard of typical curtain materials in iso 9705 fire test room,” *Fire and Materials*, vol. 36, no. 2, pp. 85–96, 2012. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1002/fam.1089?msocid=200971c6c2586ad8088b653ac3856b8d>

- [11] R. Valerdi, “Cost metrics for unmanned aerial vehicles,” in *Proceedings of the 5th Annual Conference on Systems Engineering Research (CSER)*. University of Southern California, 2005. [Online]. Available: https://www.researchgate.net/publication/255598559_Cost_Metrics_for_Unmanned_Aerial_Vehicles